

SUPSI

Cross-Test of Hard Instances for TSP Algorithms and Heuristics

Studente/i

Dominik Panzarella

Relatore

Palmo Monaldo Mastrolilli

Correlatore

Villa Tullio

Committente

Palmo Monaldo Mastrolilli

Corso di laurea

Ingegneria Informatica

Modulo

Progetto di diploma

Anno

2025

Data

21 agosto 2025

Aspettando la bellavita.

Indice

Abstract	xi
1 Introduzione	1
2 The Traveling Salesman Problem (TSP)	3
2.1 Introduzione al problema	3
2.1.1 Definizione formale del problema	3
2.2 Modellazione matematica e varianti	3
2.2.1 Il TSP come problema di teoria dei grafi	3
2.2.2 Ciclo Hamiltoniano	4
2.2.3 TSP simmetrico e asimmetrico	4
2.3 Complessità computazionale del TSP	5
2.3.1 Il TSP è davvero difficile da risolvere?	5
2.3.2 Prospettiva storica e primi approcci risolutivi	5
2.3.3 La nozione di algoritmi efficienti	6
2.3.4 Problema Decisionale	6
2.3.5 Le classi P e NP nella teoria della complessità	7
2.3.6 TSP come problema decisionale	8
2.3.7 Problemi NP-hard e NP-complete	9
3 Algoritmi e Euristiche per il TSP	11
3.1 Algoritmi Costruttivi	12
3.1.1 Nearest Neighbor	12
3.1.2 Nearest Insertion	14
3.1.3 Farthest Insertion	14
3.2 Algoritmi di Miglioramento	16
3.2.1 K-opt	16
3.2.2 Lin-Kernighan	17
3.2.3 Lin-Kernighan-Helsgaun (LKH)	19
3.3 Algoritmi esatti	20
3.3.1 Branch-and-Bound	20

3.3.2	Branch-and-Cut	22
4	Implementazione	25
4.1	Architettura del sistema	25
4.1.1	Struttura backend	25
4.2	Interfacce	26
4.3	Repository Layer	28
4.3.1	Gestione delle Istanze TSP	29
4.3.2	Lettura delle Istanze TSP	30
4.3.3	Creazione istanze TSP	31
4.3.4	Creazione dei report	31
4.4	Service Layer	32
4.4.1	Generatore Istanze difficili	32
4.4.1.1	L'importanza delle istanze difficili	33
4.4.1.2	Nearest Neighbour	34
4.4.1.3	Nearest Insertion	36
4.4.1.4	Integrality Gap	38
4.4.1.5	Concorde	39
4.4.2	Implementazione degli algoritmi	40
4.4.3	Sistema di esecuzione	41
4.5	Controller Layer	42
4.6	Inizializzazione del Sistema	43
5	Risultati Sperimentali	45
5.1	Algoritmi Testati	45
5.2	Set di Istanze Utilizzato	45
5.2.1	Istanze Standard	45
5.2.2	Generazione delle Istanze Difficili	47
5.2.3	Altre Istanze Difficili	49
5.3	Macchina utilizzata	52
5.4	Configurazione degli esperimenti	53
5.5	Visualizzazione dei risultati	54
5.5.1	Premessa su ALG/OPT e OPT	54
5.5.2	Grafici per algoritmo e classe	54
5.5.3	Grafici aggregati per fascia dimensionale	55
5.5.4	Grafici combinati per classe	55
5.5.5	Grafici combinati per algoritmo	55
5.5.6	Note aggiuntive	56
5.6	Analisi dei risultati	56
5.6.1	Grafici aggregati per fascia dimensionale	56

5.6.1.1	Dimensioni ridotte (0-50 nodi)	56
5.6.1.2	Dimensioni medie (51-200 nodi)	56
5.6.1.3	Dimensioni elevate (201-2000 nodi)	57
5.6.1.4	Dimensioni molto elevate (2001+ nodi)	58
5.6.2	Analisi Comparativa su Classi Specifiche	59
5.6.2.1	Classe HARDNI	59
5.6.2.2	Classe HARDNN	60
5.6.2.3	Classe HIGH_IG	61
5.6.2.4	Classe TSPLIB	62
5.6.2.5	Classe HARDTSPLIB	64
5.6.2.6	Classe TNM	66
5.6.3	Analisi Comparativa per Algoritmo	70
5.6.3.1	Concorde	70
5.6.3.2	LKH3	71
5.6.3.3	FarthestInsertion	71
5.6.3.4	NearestInsertion	72
5.6.3.5	NearestNeighbour	73
6	Conclusione	75
6.1	Considerazioni Finali	75
6.1.1	Implicazioni Pratiche	75
6.1.2	Prospettive di Ricerca	76

Elenco delle figure

2.1	Visualizzazione di un problema decisionale: ogni input produce un output binario. Fonte: Wikipedia, <i>Decision problem</i> (https://en.wikipedia.org/wiki/Decision_problem), licenza CC BY-SA 3.0.	7
2.2	Rappresentazione delle relazioni tra le classi di complessità $\mathcal{P}$, $\mathcal{NP}$, $\mathcal{NP}$ -completo e $\mathcal{NP}$ -hard nei due scenari: a sinistra il caso ipotetico $\mathcal{P} \neq \mathcal{NP}$, a destra $\mathcal{P} = \mathcal{NP}$	9
4.1	Diagramma UML delle classi e interfacce principali per la gestione delle configurazioni nel sistema. Vengono mostrati i livelli di astrazione tra le impostazioni generali e di istanza, nonché la relazione con il provider di configurazioni.	29
4.2	Gerarchia classi Reader	31
4.3	Gerarchia classi Writer	32
4.4	Gerarchia delle classi per la generazione di istanze difficili	33
4.5	Esempio di istanza difficile per l'algoritmo <i>Nearest Neighbour</i> . L'immagine rappresenta una distribuzione dei nodi progettata per ingannare l'euristica greedy dell'algoritmo. Fonte: adattata da [1].	34
4.6	Esempio di istanza difficile per l'euristica <i>Nearest Insertion</i> . I nodi sono disposti ai vertici di un poligono regolare con archi lungo il perimetro di costo 1 e archi "intermedi" leggermente meno costosi. Fonte: adattata da [1].	37
4.7	Esempio costruzione dell'istanza $F(a, b, c)$ per il calcolo dell'Integrality Gap. Fonte: adattata da [2].	38
4.8	Diagramma UML che mostra la gerarchia e le relazioni tra l'interfaccia <i>IAlgorithm</i> , i solver implementati e le rispettive configurazioni.	40
4.9	Diagramma UML del sistema di esecuzione, con le principali classi e interfacce coinvolte nell'orchestrazione degli algoritmi.	41
4.10	Diagramma generico che riassume la relazione tra i tre layer: Controller, Service, Repository.	42

4.11	Diagramma di sequenza del metodo <code>Initializer::init</code>	44
5.1	Performance aggregata dei solver su istanze di dimensione compresa tra 0 e 50 nodi.	56
5.2	Performance aggregata dei solver su istanze di dimensione compresa tra 51 e 100 nodi.	57
5.3	Performance aggregata dei solver su istanze di dimensione compresa tra 101 e 200 nodi.	57
5.4	Performance aggregata dei solver su istanze di dimensione compresa tra 201 e 1000 nodi.	58
5.5	Performance aggregata dei solver su istanze di dimensione compresa tra 1001 e 2000 nodi.	58
5.6	Performance aggregata dei solver su istanze di dimensione compresa tra 2001 e 5000 nodi.	59
5.7	Performance aggregata dei solver su istanze di dimensione a partire da 5001 nodi.	59
5.8	Confronto delle prestazioni sulla classe HARDNI. Si noti la divergenza tra algoritmi esatti ed euristici sia in termini di qualità (alto) che di tempo di esecuzione (basso).	60
5.9	Prestazioni sulla classe HARDNN. Evidente il comportamento non monotono del Nearest Neighbour (linea rossa) nella metrica ALG/OPT.	61
5.10	Prestazioni sulla classe HIGH_IG. Si osservi l'anomalia temporale di LKH3 (linea verde) e le prestazioni variabili di NearestInsertion (linea viola).	61
5.11	Dettaglio delle prestazioni di Nearest Insertion su HIGH_IG. Le ampie barre di errore evidenziano l'alta variabilità dei risultati.	62
5.12	Performance aggregata dei solver su istanze TSPLIB.	63
5.13	Performance LKH3 su istanze TSPLIB.	63
5.14	Performance Concorde su istanze TSPLIB.	64
5.15	Performance Farthest Insertion su istanze TSPLIB.	64
5.16	Performance Nearest Neighbour su istanze TSPLIB.	64
5.17	Performance Nearest Insertion su istanze TSPLIB.	65
5.18	Performance aggregata dei solver su istanze HARDTSPLIB.	66
5.19	Performance Concorde su istanze HARDTSPLIB.	66
5.20	Performance LKH3 su istanze HARDTSPLIB.	66
5.21	Performance Nearest Insertion su istanze HARDTSPLIB.	67
5.22	Performance Nearest Neighbour su istanze HARDTSPLIB.	67
5.23	Performance Farthest Insertion su istanze HARDTSPLIB.	67

5.24 Confronto aggregato delle prestazioni degli algoritmi sulle istanze TNM. I tempi di esecuzione (sinistra) e i valori di ALG/OPT (destra) mostrano il trade-off tra precisione e velocità.	68
5.25 Prestazioni di Concorde su istanze TNM. Si noti la stabilità dei valori ALG/OPT (alto) e la crescita lineare dei tempi (basso).	69
5.26 Prestazioni di LKH3 su istanze TNM. Osservare l'andamento esponenziale dei tempi di esecuzione (basso) nonostante la precisione ottimale (alto). . . .	69
5.27 Prestazioni di Nearest Insertion su istanze TNM. Notevole la maggiore variabilità nei valori ALG/OPT rispetto a FarthestInsertion.	69
5.28 Prestazioni di Nearest Neighbour su istanze TNM. Tempi di esecuzione trascurabili (sinistra) ma qualità delle soluzioni inferiori (destra).	70
5.29 Prestazioni di Farthest Insertion su istanze TNM. Si apprezzi il miglior compromesso tra qualità e velocità tra le euristiche.	70
5.30 Prestazioni di Concorde su tutte le classi di istanze. Notare la scala logaritmica per i tempi di esecuzione (sinistra) e la stabilità dell'ALG/OPT (destra). .	71
5.31 Prestazioni di LKH3. Si notino le oscillazioni nei tempi di esecuzione non correlate alla dimensione.	71
5.32 Prestazioni di FarthestInsertion. Costante rapporto qualità/velocità.	72
5.33 Prestazioni di NearestInsertion. Notevole variabilità nell'ALG/OPT.	72
5.34 Prestazioni di NearestNeighbour. Tempi trascurabili ma qualità compromessa.	73

Elenco delle tabelle

5.1	Istanze simmetriche TSP da <i>TSPLIB95</i>	45
5.2	Istanze generate per il caso peggiore di Nearest Neighbor (<i>HARDNN</i>)	48
5.3	Istanze generate per il caso peggiore di Nearest Insertion (<i>HARDNI</i>)	48
5.4	Istanze basate sulla costruzione di Benoit e Boyd per massimizzare l'integrality gap (<i>HIGH_IG</i>)	48
5.5	Istanze libreria <i>HARDTSPLIB</i>	49
5.6	Istanze della famiglia <i>Tnm</i>	51

Abstract

English

This project investigates the performance of various algorithms and heuristics on hard instances of the Travelling Salesman Problem (TSP). The main goal is to perform a cross-test comparison of exact solvers and approximate methods, analyzing their strengths and weaknesses in dealing with particularly challenging problem instances.

The study covers classical algorithms such as Concorde and LKH3, as well as heuristics like Nearest Neighbor and Christofides. Experimental results highlight the trade-offs between solution quality and computational time, providing insights into the suitability of different methods depending on instance characteristics.

This work contributes to the understanding of TSP solvers' behavior on difficult inputs, aiming to guide future research and practical applications in combinatorial optimization.

Italian

Questo progetto analizza le prestazioni di diversi algoritmi ed euristiche su istanze difficili del Problema del Commesso Viaggiatore (**TSP**, dall' inglese "*Traveling Salesman Problem*"). L'obiettivo principale è effettuare un confronto incrociato tra solver esatti e metodi approssimati, analizzandone punti di forza e debolezza nel trattare casi particolarmente complessi.

Lo studio prende in esame algoritmi classici come Concorde e LKH3, oltre a euristiche come Nearest Neighbor e Christofides. I risultati sperimentali evidenziano i compromessi tra qualità della soluzione e tempo di calcolo, fornendo indicazioni sull'adeguatezza dei diversi metodi in base alle caratteristiche delle istanze.

Questo lavoro contribuisce alla comprensione del comportamento dei solver per il TSP su input difficili, con l'intento di orientare la ricerca futura e le applicazioni pratiche nell'ambito dell'ottimizzazione combinatoria.

Capitolo 1

Introduzione

Il *Problema del Commesso Viaggiatore* (TSP – *Traveling Salesman Problem* in inglese) rappresenta uno dei problemi più studiati nell'ambito dell'ottimizzazione combinatoria e della teoria della complessità computazionale. Nonostante la sua formulazione sia estremamente semplice – trovare il ciclo di costo minimo che visita ciascuna città una sola volta e ritorna al punto di partenza – la sua risoluzione risulta intrattabile dal punto di vista computazionale, in quanto classificato come problema NP-hard.

Il TSP è diventato un banco di prova per lo sviluppo di algoritmi esatti, euristiche e metaeuristiche, proprio per queste sue caratteristiche. Le sue applicazioni si estendono ben oltre il contesto teorico: dalla logistica alla bioinformatica, dal design di microchip all'intelligenza artificiale.

Negli ultimi decenni sono stati proposti numerosi algoritmi per affrontare il problema, ciascuno con punti di forza e debolezza che si manifestano in funzione della struttura dell'istanza da risolvere. Alcune istanze si sono dimostrate particolarmente sfidanti anche per i metodi più sofisticati, spingendo i ricercatori a esplorare il concetto di *istanze difficili*: configurazioni in grado di mettere in crisi molteplici risolutori.

Questo progetto nasce con l'obiettivo di analizzare criticamente le prestazioni di diversi algoritmi ed euristiche su un insieme selezionato di *istanze difficili* del TSP, identificate tramite generatori specifici e approcci noti in letteratura. Gli obiettivi sono due: da un lato, valutare comparativamente la robustezza e i limiti degli approcci considerati; dall'altro, contribuire alla comprensione delle caratteristiche che rendono un'istanza difficile per una determinata categoria di algoritmi.

Per raggiungere questo scopo, è stato sviluppato un *sistema software modulare* che consente l'esecuzione automatizzata e configurabile di molteplici algoritmi su insiemi di istanze, con raccolta e analisi dei risultati. In particolare, il sistema include algoritmi esatti (come Concorde), algoritmi di miglioramento (come LKH3) e costruttivi (come Nearest Neighbor,

Nearest Insertion, Farthest Insertion), oltre a un framework per la generazione delle **istanze difficili**.

La struttura del lavoro è la seguente:

- Il **Capitolo 2** introduce formalmente il TSP, presentandone le varianti, la modellazione matematica e la complessità computazionale.
- Il **Capitolo 3** analizza le principali famiglie di algoritmi per il TSP: costruttivi, di miglioramento e metodi esatti.
- Il **Capitolo 4** descrive l'implementazione del **sistema software** sviluppato, con un particolare focus sull'architettura e sulle sue funzionalità
- Il **Capitolo 5** presenta e discute i risultati sperimentali ottenuti attraverso test incrociati su istanze difficili.
- Il **Capitolo 6** riassume le conclusioni e propone possibili sviluppi futuri.

Attraverso questo lavoro si intende fornire un contributo alla comprensione delle dinamiche che governano l'efficacia degli algoritmi per il TSP, offrendo un quadro sistematico utile sia a fini di ricerca che per applicazioni pratiche.

Capitolo 2

The Traveling Salesman Problem (TSP)

2.1 Introduzione al problema

2.1.1 Definizione formale del problema

Il problema del commesso viaggiatore (TSP, dall'inglese Traveling Salesman Problem) richiede di determinare il percorso più economico che permetta di visitare una serie di città, ciascuna una sola volta, e di tornare infine al punto di partenza. In questo contesto, il termine percorso indica una specifica sequenza in cui le città vengono attraversate, detta **tour** o **circuito**. [3]

Il TSP è diventato uno dei rari problemi matematici moderni a entrare nell'immaginario collettivo. Oltre al nome suggestivo, ciò che ha realmente catturato l'interesse della comunità scientifica e non solo è la combinazione tra la chiarezza della sua formulazione e la difficoltà nel trovare una **soluzione generale**. Proprio per questa dualità, il TSP rappresenta un banco di prova privilegiato per lo sviluppo e la sperimentazione di metodi risolutivi nell'ambito dell'ottimizzazione e della teoria della computazione. [3]

2.2 Modellazione matematica e varianti

2.2.1 Il TSP come problema di teoria dei grafi

Il problema del commesso viaggiatore (TSP) si può interpretare in termini di teoria dei grafi come la ricerca di un **ciclo hamiltoniano di costo minimo** all'interno di un grafo pesato. In questa rappresentazione, ogni città corrisponde a un *vertice* del grafo $G = (V, E)$, mentre i possibili percorsi tra le città sono modellati come *archi* che collegano tali vertici. A ciascun

arco $(i, j) \in E$ viene associato un *peso* c_{ij} , che può indicare il costo, la distanza o il tempo necessario per spostarsi tra le due città collegate. [2] [4]

In molti casi, si considera il grafo completo, cioè con archi tra tutte le coppie di vertici. Se alcune città non sono collegate direttamente, si possono inserire archi artificiali con pesi molto elevati per garantire la completezza senza influenzare il percorso ottimale.

Un'alternativa più raffinata consiste nell'applicare il **completamento metrico**, che assegna a ogni coppia di città la distanza del percorso *più breve* tra di esse. Questo approccio preserva la struttura metrica del problema, evitando di introdurre valori arbitrariamente grandi. [3]

L'obiettivo finale del TSP consiste nel trovare un ciclo chiuso di costo minimo che soddisfi due requisiti fondamentali:

- iniziare e terminare nello stesso vertice, corrispondente alla città di partenza;
- visitare ogni altro vertice esattamente una volta, senza ripetizioni.

2.2.2 Ciclo Hamiltoniano

Un **ciclo hamiltoniano** in un grafo non orientato è un ciclo semplice che visita ciascun vertice esattamente una volta, tornando infine al punto di partenza. Il problema di determinare se un grafo contiene un tale ciclo è noto come *problema del ciclo hamiltoniano*, ed è uno dei problemi fondamentali della teoria dei grafi [5] [6].

Il problema è noto per essere **NP-completo**, come dimostrato da Karp nel suo lavoro del 1972 [7] [8], ciò implica che non si conosce un algoritmo efficiente per risolverlo in generale, sebbene esistano approcci esatti o euristici per casi specifici.

2.2.3 TSP simmetrico e asimmetrico

Il TSP si presenta in due principali varianti a seconda della struttura del grafo sottostante:

- **TSP simmetrico (STSP)**: è il caso in cui $c_{ij} = c_{ji}$ per ogni coppia di città i, j . Questo implica che il grafo è *non orientato* e i costi di percorrenza tra due città sono identici in entrambe le direzioni. La simmetria riduce lo spazio delle soluzioni possibili, in quanto ogni tour ha un equivalente inverso di pari costo. [2] [4]
- **TSP asimmetrico (ATSP)**: in questo caso, $c_{ij} \neq c_{ji}$ per almeno una coppia di città, dando luogo a un *grafo orientato*. Le differenze nei costi possono derivare da condizioni reali come il traffico, strade a senso unico o differenze tariffarie nei voli (es. costi di andata e ritorno non simmetrici). Questa variante è generalmente più complessa da risolvere a causa della maggiore dimensione dello spazio delle soluzioni. [2] [4]

Dal punto di vista computazionale, entrambe le varianti sono *NP-hard* (come verrà approfondito in seguito nella sezione 2.3.7), ma il TSP asimmetrico tende a essere più difficile da trattare, richiedendo tecniche di modellazione e algoritmi di risoluzione specifici.

2.3 Complessità computazionale del TSP

2.3.1 Il TSP è davvero difficile da risolvere?

Quando una descrizione del TSP appare in un contesto popolare, è solitamente accompagnata da una breve descrizione della notoria difficoltà del problema. **Ma è vero che il TSP è effettivamente difficile da risolvere?** La verità è che *non lo sappiamo con certezza*.

2.3.2 Prospettiva storica e primi approcci risolutivi

Il problema del commesso viaggiatore ha una lunga storia alle sue spalle. Le prime formulazioni intuitive risalgono al XIX secolo, ma è solo nel XX secolo che il problema viene formalizzato in modo rigoroso.

Nel **1832**, il matematico irlandese William Rowan Hamilton propose un gioco chiamato *Icosian Game*, che consisteva nel trovare un ciclo che attraversasse una serie di città disposte secondo un dodecaedro regolare. Anche se non formulato esplicitamente come problema di ottimizzazione, esso è riconosciuto come uno dei primi esempi di **ciclo hamiltoniano**, concetto chiave nel TSP [9].

Nel 1931, il matematico austriaco Karl Menger discusse il *Botenproblem* (problema del messaggero), proponendolo come problema di interesse teorico e osservando già allora la complessità insita nella ricerca del tour più breve tra un insieme di città [10]. Traducendo le sue osservazioni:

Questo problema può naturalmente essere risolto utilizzando un numero finito di tentativi. Non sono note regole che riducano il numero di tentativi al di sotto del numero di permutazioni dell'insieme di punti dato. La regola secondo cui si dovrebbe andare dal punto di partenza al punto più vicino, poi al successivo punto più vicino, e così via, non produce sempre il percorso più breve.

Menger osservò dunque che è possibile risolvere il TSP semplicemente controllando ogni tour, uno dopo l'altro, e scegliendo quello più economico. Tuttavia, tale approccio ha complessità $(n - 1)!$ e diventa rapidamente impraticabile anche per istanze di dimensioni moderate.

Negli anni '50, il problema iniziò a guadagnare interesse pratico, specialmente nell'ambito della logistica e dell'organizzazione industriale. La compagnia Rand Corporation iniziò a

studiarlo nel contesto dell'ottimizzazione militare. Nel 1954, il matematico George Dantzig, affrontò un'istanza del TSP con 49 città statunitensi, risolvendola con i primi calcolatori numerici. Questo evento segnò l'inizio del trattamento computazionale sistematico del TSP [11].

Nel 1964, Gilmore e Gomory pubblicarono un articolo intitolato "*A solvable case of the traveling salesman problem*" [12], che analizzava casi speciali del TSP risolvibili in modo più efficiente, sottolineando così l'inadeguatezza dell'approccio esaustivo.

In quegli stessi anni, si svilupparono le prime versioni di algoritmi euristici e meta-euristici, come il metodo del *nearest neighbor* e le varianti del *2-opt*. Tuttavia, fu solo con la formalizzazione della teoria della complessità negli anni '70 che il TSP trovò la sua piena collocazione teorica.

In particolare, il lavoro di Stephen Cook (1971) [13] e Richard Karp (1972) [7] permise di classificare il TSP come **problema NP-hard**, evidenziando il suo ruolo centrale nella frontiera tra problemi trattabili e intrattabili.

Oggi, il TSP è considerato un problema paradigmatico non solo per la ricerca operativa e l'ottimizzazione combinatoria, ma anche per la teoria della complessità computazionale, la matematica applicata e l'intelligenza artificiale.

2.3.3 La nozione di algoritmi efficienti

Un punto di svolta nella comprensione teorica della complessità del problema del commesso viaggiatore (TSP) avvenne negli anni '60 grazie al lavoro pionieristico di Jack Edmonds [14]. Egli fu tra i primi a proporre una definizione formale di algoritmo **efficiente**, introducendo il concetto di *good algorithm*, cioè un algoritmo che può essere eseguito in tempo polinomiale rispetto alla dimensione dell'input.

Per appartenere a questa categoria, un algoritmo deve avere una garanzia $f(n)$ che sia una funzione polinomiale della dimensione del problema n , ovvero, per valori sufficientemente grandi di n , l'algoritmo deve terminare in un tempo al più pari a Kn^c per alcune costanti $K > 0$ e $c > 0$. Si noti che le costanti K e c devono essere fisse per ogni n sufficientemente grande. La costante di proporzionalità K viene spesso omessa nella descrizione del tempo di esecuzione, utilizzando la notazione standard $\mathcal{O}(n^c)$ per indicare il vincolo asintotico sul tempo di esecuzione.

2.3.4 Problema Decisionale

In ambito teorico, un **problema decisionale** è un tipo particolare di problema computazionale il cui output, per ogni input fornito, è una risposta binaria: *sì* oppure *no*. Questo tipo di problemi è centrale sia nella *teoria della computabilità* che nella *teoria della complessità computazionale*. [15] [16]

Un **algoritmo decisionale** è un procedimento sistematico in grado di restituire sempre una risposta affermativa o negativa per ogni possibile input. Se tale algoritmo esiste per un dato problema, il problema si dice **decidibile**. [15] [16]

Un esempio grafico di problema decisionale è mostrato in figura 2.1, dove viene evidenziata la struttura a due esiti possibili (YES/NO), a partire da un generico input.

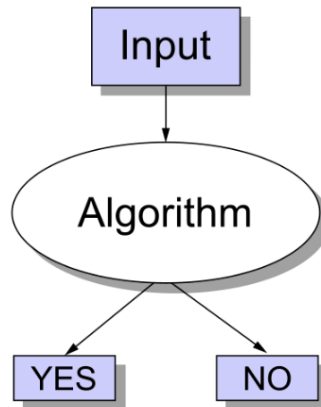


Figura 2.1: Visualizzazione di un problema decisionale: ogni input produce un output binario.

Fonte: Wikipedia, *Decision problem* (https://en.wikipedia.org/wiki/Decision_problem), licenza CC BY-SA 3.0.

2.3.5 Le classi P e NP nella teoria della complessità

Una delle domande fondamentali emerse con la formalizzazione della complessità computazionale è: *esiste un algoritmo efficiente per risolvere il problema del commesso viaggiatore (TSP)?* Questo interrogativo fu sollevato già da Jack Edmonds, ma ancora oggi non ha ricevuto una risposta definitiva. L'importanza di questa domanda è tale che il *Clay Mathematics Institute* ha incluso la questione più generale $\mathcal{P} \stackrel{?}{=} \mathcal{NP}$ tra i *Millennium Prize Problems*, offrendo un premio di un milione di dollari per una sua risoluzione formale [17].

La definizione dei problemi decisionali è utile per classificare i problemi in base alla loro difficoltà computazionale. All'interno di questo quadro teorico, le classi principali sono:

- $\mathcal{P}$ (*Polynomial Time*): l'insieme dei problemi che possono essere risolti in tempo polinomiale tramite un algoritmo deterministico. Si tratta di problemi per cui esistono algoritmi considerati efficienti secondo la definizione di Edmonds [14] [18] [19].
- $\mathcal{NP}$ (*Nondeterministic Polynomial Time*): include i problemi per cui, se la risposta al problema decisionale è "sì", esiste un *certificato* (cioè una soluzione) che può essere verificato in tempo polinomiale da un algoritmo deterministico [13] [18] [19].

Il nodo centrale della teoria è determinare se le due classi coincidano:

$$\mathcal{P} \stackrel{?}{=} \mathcal{NP}$$

La questione centrale nella teoria della complessità computazionale riguarda la relazione tra le classi $\mathcal{P}$ e $\mathcal{NP}$, che si fonda su una distinzione fondamentale tra la *verifica* e la *risoluzione* di un problema decisionale. I problemi appartenenti alla classe $\mathcal{NP}$ sono caratterizzati dalla proprietà che, qualora la risposta sia affermativa, esiste un certificato la cui validità può essere verificata in tempo polinomiale da un algoritmo deterministico [13] [18]. Viceversa, la classe $\mathcal{P}$ comprende i problemi per i quali esiste un algoritmo deterministico in grado di trovare una soluzione (o determinare la risposta corretta) in tempo polinomiale rispetto alla dimensione dell'input [14] [19].

L'ipotesi $\mathcal{P} = \mathcal{NP}$ affermerebbe quindi che ogni problema la cui soluzione può essere verificata in modo efficiente possa altresì essere risolto in modo altrettanto efficiente, ovvero che la complessità computazionale necessaria per la verifica e per la ricerca di una soluzione siano equivalenti [18]. Ciò implicherebbe una rivoluzione nei campi della matematica, dell'informatica e delle scienze applicate, poiché consentirebbe di risolvere in maniera efficiente un vasto insieme di problemi fino ad oggi considerati intrattabili. Le due ipotesi opposte, $\mathcal{P} = \mathcal{NP}$ e $\mathcal{P} \neq \mathcal{NP}$, sono rappresentate schematicamente nella Figura 2.3.5, che mostra le relazioni tra le principali classi di complessità, tra cui $\mathcal{P}$, $\mathcal{NP}$, $\mathcal{NP}$ -completo e $\mathcal{NP}$ -hard.

Tuttavia, la maggioranza della comunità scientifica ritiene che $\mathcal{P} \neq \mathcal{NP}$, in virtù della mancanza di metodi generali per trasformare una verifica efficiente in una risoluzione efficiente e del fatto che molti problemi in $\mathcal{NP}$ mostrano una complessità computazionale empiricamente molto più elevata nella fase di ricerca della soluzione rispetto a quella di verifica [7] [18] [19]. Tale convinzione si basa su decenni di studi teorici e sperimentali che evidenziano l'enorme divario tra la difficoltà di trovare una soluzione ottimale e quella di verificarne una già fornita.

2.3.6 TSP come problema decisionale

Il problema del commesso viaggiatore (TSP), nella sua formulazione classica, è un problema di **ottimizzazione**: si richiede di trovare un ciclo di costo minimo che visiti ogni città esattamente una volta. Tuttavia, ai fini della teoria della complessità, il TSP può essere riformulato come un **problema decisionale** [2] [15].

La versione decisionale del TSP è la seguente:

Dato un grafo completo pesato $G = (V, E)$ e una soglia $K \in \mathbb{R}$, esiste un tour hamiltoniano di costo totale $\leq K$?

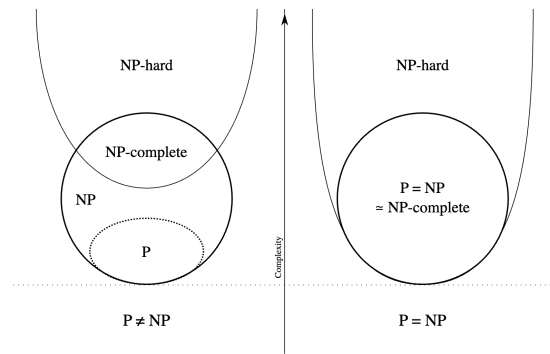


Figura 2.2: Rappresentazione delle relazioni tra le classi di complessità $\mathcal{P}$, $\mathcal{NP}$, $\mathcal{NP}$ -completo e $\mathcal{NP}$ -hard nei due scenari: a sinistra il caso ipotetico $\mathcal{P} \neq \mathcal{NP}$, a destra $\mathcal{P} = \mathcal{NP}$.

Fonte: Wikipedia, *P vs NP problem*

(https://en.wikipedia.org/wiki/P_versus_NP_problem), licenza CC BY-SA 3.0.

Questa riformulazione permette di analizzare il TSP all'interno del quadro teorico delle classi $\mathcal{P}$ e $\mathcal{NP}$. In particolare, se per una data istanza la risposta è "sì", è possibile fornire un *certificato* — ovvero il tour stesso — e verificarne la correttezza in tempo polinomiale controllando che:

- il tour includa tutte le città, ciascuna una sola volta;
- il costo complessivo del tour non superi la soglia K .

Questo processo di verifica in tempo polinomiale dimostra che la versione decisionale del TSP appartiene alla classe $\mathcal{NP}$. Se si scoprisse un algoritmo in grado di risolvere tale problema decisionale in tempo polinomiale per ogni possibile istanza, ciò implicherebbe che $\mathcal{P} = \mathcal{NP}$ [2].

2.3.7 Problemi NP-hard e NP-complete

Un problema decisionale C è detto **NP-completo** se soddisfa entrambe le seguenti condizioni:

1. $C \in \mathcal{NP}$, cioè una soluzione candidata per C può essere verificata in tempo polinomiale [13] [19];
2. Ogni problema in $\mathcal{NP}$ è riducibile a C in tempo polinomiale (riduzione polinomiale) [7] [18].

Un problema che soddisfa soltanto la seconda condizione, senza appartenere necessariamente a $\mathcal{NP}$, è detto **NP-hard** [18].

Una conseguenza importante di queste definizioni è che, se esistesse un algoritmo in grado di risolvere un problema NP-completo in tempo polinomiale (su una Macchina di Turing

universale o un modello computazionale equivalente), allora tutti i problemi della classe NP potrebbero essere risolti in tempo polinomiale [19] [18].

Il TSP, nella sua versione decisionale, è NP-completo, come dimostrato dai risultati fondamentali di Cook [13] e Karp [7].

Pertanto, come già discusso, se esistesse un algoritmo capace di risolverlo in tempo polinomiale, si avrebbe

$$\mathcal{P} = \mathcal{NP},$$

$\mathcal{P}=\mathcal{NP}$, con profonde implicazioni per la teoria della complessità computazionale e numerosi ambiti applicativi [18] [19].

Capitolo 3

Algoritmi e Euristiche per il TSP

La risoluzione del Traveling Salesman Problem (TSP) ha dato origine a una vasta gamma di approcci algoritmici, ciascuno con caratteristiche, vantaggi e limitazioni specifiche. La natura NP-hard del problema [7] [8] [18] (approfondita nella sezione 2.3.7) ha spinto i ricercatori a sviluppare strategie diverse per affrontare la sfida computazionale, bilanciando qualità della soluzione e tempo di calcolo.

Gli algoritmi per il TSP possono essere classificati in tre categorie principali, ognuna delle quali rappresenta una filosofia di approccio diversa al problema:

- **Algoritmi Costruttivi** rappresentano la strategia più immediata: costruiscono una soluzione valida partendo da zero, aggiungendo iterativamente città al tour fino al completamento. Questi metodi risolutivi sono caratterizzati da **semplicità implementativa** e **velocità di esecuzione**, rendendoli ideali per ottenere rapidamente una soluzione di partenza. Tuttavia, essendo algoritmi greedy, tendono a rimanere intrappolati in ottimi locali, producendo soluzioni spesso distanti dall'ottimo globale [1] [20].
- **Algoritmi di Miglioramento** adottano una strategia complementare: partono da una soluzione esistente che viene successivamente perturbata attraverso modifiche locali. Questi algoritmi sfruttano il concetto di *ricerca locale*, esplorando sistematicamente il vicinato (neighborhood) di una soluzione per identificare miglioramenti. Le varianti k-opt e la euristica di Lin-Kernighan [20] [21] rappresentano alcuni tra i metodi più efficaci. La loro efficacia dipende fortemente dalla qualità della soluzione iniziale e dalla definizione appropriata delle operazioni di esplorazione del vicinato.
- **Algoritmi Esatti** garantiscono il raggiungimento della soluzione ottima attraverso un'esplorazione sistematica dello spazio delle soluzioni. Utilizzano tecniche sofisticate di branch-and-bound e branch-and-cut [3] [11]. Sebbene teoricamente superiori, la loro applicabilità è limitata dalle dimensioni del problema a causa della crescita esponenziale del tempo di calcolo.

La scelta dell'algoritmo più appropriato dipende da diversi fattori: dimensione dell'istanza, qualità richiesta della soluzione, tempo disponibile per il calcolo e caratteristiche specifiche del problema. In molte applicazioni pratiche, si adotta un approccio ibrido che combina la velocità degli algoritmi costruttivi con la precisione degli algoritmi di miglioramento [2].

Nel contesto di questo progetto, l'analisi comparativa di questi diversi approcci su *istanze difficili* permetterà di identificare **pattern di comportamento** e **caratteristiche specifiche** che influenzano le performance degli algoritmi. Questa comprensione è fondamentale per rispondere alla domanda centrale del nostro lavoro: **esistono istanze universalmente difficili per tutti gli algoritmi?**

Nei prossimi paragrafi esamineremo in dettaglio ciascuna categoria, analizzando i principi teorici, le caratteristiche implementative e le performance attese di ogni algoritmo selezionato per il nostro studio sperimentale.

3.1 Algoritmi Costruttivi

Nel contesto dell'analisi delle prestazioni, si indica comunemente con **OPT** il costo del *tour ottimale* per una data istanza del problema. Quando si parla di rapporto di approssimazione o qualità della soluzione, ci si riferisce quindi al rapporto tra il costo del tour prodotto da un algoritmo e OPT.

3.1.1 Nearest Neighbor

Descrizione

L'algoritmo *Nearest Neighbor* (NN) è uno dei più antichi e semplici approcci euristici sviluppati per affrontare il *Travelling Salesman Problem* (TSP). Rappresenta un tipico esempio di strategia **greedy**, in cui a ogni iterazione si seleziona la scelta localmente ottimale nella speranza di ottenere una buona soluzione globale. L'idea alla base è intuitiva: partendo da una città arbitraria, si visita iterativamente la città non ancora visitata più vicina, fino a completare un tour [1].

Algoritmo

L'algoritmo può essere formalizzato nei seguenti passaggi:

1. Selezionare un nodo iniziale v_0 arbitrario.
2. Fissare v_0 come nodo corrente. Tra tutti i nodi non ancora visitati, selezionare quello più vicino (cioè con la distanza minima da v_0), e aggiungerlo al tour.
3. Impostare il nodo selezionato come nuovo nodo corrente e ripetere il passo 2.

4. Quando tutte le città sono state visitate, ritornare alla città iniziale per chiudere il tour.

Caratteristiche

L'algoritmo *Nearest Neighbour (NN)* è apprezzato per la sua semplicità e per il basso costo computazionale. In una implementazione naïve ha complessità $O(n^2)$, ma può essere ottimizzato a $O(n \log n)$ utilizzando strutture dati efficienti per la gestione delle distanze [24]. Tuttavia, presenta numerose limitazioni:

- La qualità della soluzione dipende fortemente dalla **scelta del nodo iniziale**. Tour costruiti a partire da nodi diversi possono differire sensibilmente in lunghezza.
- Il comportamento *greedy* impedisce all'algoritmo di correggere decisioni subottimali prese nelle fasi iniziali, rendendolo vulnerabile agli **ottimi locali**.
- Il **rapporto di approssimazione** nel caso pessimo è noto essere logaritmico. In particolare, Rosenkrantz et al. [?] mostrano che:

$$\frac{NN}{OPT} \leq \frac{1}{2} (\lfloor \log_2 n \rfloor + 1)$$

dove n è il numero di città. Questo rappresenta un *bound superiore*: in pratica, la qualità della soluzione può essere molto migliore.

- Esistono istanze costruite ad hoc in cui il rapporto $NN/OPT \sim \log n$, a dimostrazione che questo bound può effettivamente essere raggiunto in casi pessimi.

Variazioni e Miglioramenti

Una modifica comune consiste nell'applicare l'algoritmo **da ciascun nodo come punto di partenza** e selezionare il tour migliore tra quelli generati. Questo approccio, noto come *repeated nearest neighbor*, fornisce una migliore robustezza in media, ma non migliora il bound teorico nel caso peggiore [24].

Ulteriori estensioni includono l'uso del NN come *soluzione iniziale* per algoritmi di miglioramento locale, come il 2-opt o la euristica di Lin-Kernighan, con lo scopo di raffinare rapidamente tour subottimali [20] [21].

Osservazioni Finali

Nonostante le sue debolezze teoriche, l'algoritmo Nearest Neighbor conserva una certa rilevanza applicativa. In particolare, viene spesso utilizzato in scenari in cui è necessario ottenere rapidamente una soluzione iniziale plausibile, oppure come baseline per valutare metodi più sofisticati. Inoltre, la sua semplicità lo rende utile in contesti didattici e come primo passo nella progettazione di pipeline di ottimizzazione più complesse [3].

3.1.2 Nearest Insertion

Descrizione

L'algoritmo Nearest Insertion (NI) è una classica euristica costruttiva sviluppata per risolvere il Travelling Salesman Problem (TSP). A differenza del Nearest Neighbor, che costruisce un percorso aperto visitando il nodo più vicino a quello corrente, NI mantiene sempre un tour parziale chiuso e vi inserisce iterativamente i nodi rimanenti.

A ogni iterazione, l'algoritmo seleziona il nodo *non ancora visitato più vicino* a uno qualsiasi dei nodi già presenti nel tour corrente. Una volta selezionato, il nodo viene inserito nella posizione del tour che comporta il *minore incremento del costo complessivo*.

Algoritmo

L'algoritmo può essere descritto come segue:

1. Inizializzare il tour con un nodo casuale.
2. Tra tutti i nodi non ancora inseriti, scegliere il nodo r più **vicino** a un qualsiasi nodo già nel tour parziale.
3. Individuare l'arco (i, j) del tour in cui l'inserimento del nodo r tra i e j comporta il minimo aumento di costo, ovvero minimizza $c_{ir} + c_{rj} - c_{ij}$.
4. Inserire r tra i e j nel tour.
5. Ripetere i passi 2–5 finché tutti i nodi non sono stati inseriti nel tour.

Caratteristiche

L'algoritmo Nearest Insertion è più sofisticato del *Nearest Neighbor*, poiché tiene conto non solo della distanza tra nodi, ma anche dell'effetto dell'inserimento nel contesto del tour globale. Garantisce un tempo di esecuzione di $O(n^2)$, simile al NN, ma con soluzioni generalmente di qualità superiore.

È stato dimostrato che l'algoritmo ha un fattore di approssimazione di 2:

$$\frac{\text{NI}}{\text{OPT}} \leq 2$$

ovvero, la lunghezza del tour ottenuto non supera mai il doppio di quella del tour ottimo [1].

3.1.3 Farthest Insertion

Descrizione

L'algoritmo *Farthest Insertion* (FI) rappresenta una variante dell'approccio inseritivo al problema del *Travelling Salesman Problem* (TSP). A differenza del *Nearest Insertion*, che pri-

vilegia i nodi più vicini al tour parziale, FI seleziona a ogni iterazione il nodo più distante dal tour corrente, in base alla distanza minima da qualsiasi nodo già inserito. Questa strategia mira a catturare sin da subito la struttura globale del problema, assegnando priorità ai nodi periferici per evitare che vengano mal posizionati nelle fasi finali della costruzione [24].

Algoritmo

L'algoritmo può essere descritto come segue:

1. Inizializzare il tour con nodo casuale.
2. Tra tutti i nodi non ancora inseriti, scegliere il nodo r più **lontano** a un qualsiasi nodo già nel tour parziale.
3. Individuare l'arco (i, j) del tour in cui l'inserimento del nodo r tra i e j comporta il minimo aumento di costo, ovvero minimizza $c_{ir} + c_{rj} - c_{ij}$.
4. Inserire r tra i e j nel tour.
5. Ripetere i passi 2–5 finché tutti i nodi non sono stati inseriti nel tour.

Caratteristiche

L'algoritmo FI possiede alcune caratteristiche distintive:

- La sua complessità computazionale è $O(n^2)$, in linea con altre euristiche costruttive.
- La strategia di inserimento dei nodi più distanti favorisce una distribuzione spaziale equilibrata del tour, riducendo il rischio di lunghi archi aggiunti a posteriori.
- In molte istanze pratiche fornisce soluzioni più accurate rispetto a metodi come il *Nearest Neighbor* o il *Nearest Insertion*.
- È particolarmente efficace in scenari in cui i nodi marginali, se trascurati nelle prime fasi, possono generare soluzioni di scarsa qualità.

Comportamento nel Caso Peggior

Sebbene l'euristica FI si comporti bene nella pratica, i risultati teorici ne limitano le garanzie nel caso peggiore. In particolare, Rosenkrantz et al. [1] hanno mostrato che la lunghezza del tour prodotto può essere al massimo:

$$\frac{\text{FI}}{\text{OPT}} \leq 2 \ln(n) + 0.16$$

dove n rappresenta il numero di nodi. Tale bound è logaritmico, risultando meno favorevole rispetto al bound costante di altre euristiche (es. 2 per *Nearest Insertion*).

3.2 Algoritmi di Miglioramento

3.2.1 K-opt

Descrizione

L'algoritmo k -opt rappresenta una famiglia di euristiche di miglioramento locale per il TSP. L'idea di base è rimuovere k archi da un tour e ricollegare i segmenti rimanenti in modo da ottenere un nuovo tour con costo inferiore. Gli algoritmi k -opt sono fondamentali nella letteratura euristica in quanto sono spesso utilizzati per costruire strategie più avanzate come LK e LKH [3] [20] .

Algoritmo

L'approccio può essere riassunto nei seguenti passi:

1. Si parte da un tour iniziale T .
2. Si selezionano k archi da rimuovere.
3. Si considerano tutte (o una parte) delle possibili modalità per ricollegare i $2k$ estremi rimanenti, al fine di formare un nuovo tour **valido**, ossia un ciclo semplice che visita ciascun nodo una sola volta.
4. Se esiste almeno un ricollegamento che riduce il costo totale del tour, lo si applica e si ripete il processo.
5. Il procedimento si arresta quando nessuna mossa k -opt consente un miglioramento ulteriore.

[21]

Un **ricollegamento valido** è una riorganizzazione degli archi rimanenti tale che il risultato sia ancora un tour connesso, ovvero un ciclo hamiltoniano.

Caratteristiche

- Il numero di configurazioni da esplorare cresce rapidamente con k : già con $k = 3$ si hanno $O(n^3)$ combinazioni.
- Nella pratica, si preferisce utilizzare valori piccoli di k (tipicamente 2 o 3).
- Le versioni più sofisticate limitano la ricerca a combinazioni promettenti usando strutture ausiliarie.

Comportamento nel Caso Peggior

Nel caso peggiore, ogni mossa k -opt richiede l'esplorazione di $\mathcal{O}(n^k)$ configurazioni, dove n è il numero di nodi e k il numero di archi rimossi e reinseriti. Inoltre, il problema di trovare un ottimo locale nella k -opt neighborhood è **PLS-completo** [18], il che implica che:

- Non esistono algoritmi noti, nemmeno randomizzati, che garantiscano la convergenza a un ottimo locale in tempo polinomiale su tutte le istanze.
- Per istanze opportunamente costruite, il numero di passi necessari per raggiungere un ottimo locale può essere esponenziale rispetto al numero di nodi.

3.2.2 Lin-Kernighan

Descrizione

L'euristica di *Lin-Kernighan* (LK), proposta nel 1973 da Shen Lin e Brian Kernighan [20], è considerata una delle euristiche più potenti per il problema del commesso viaggiatore (TSP). A differenza degli algoritmi k -opt standard, in cui il numero di archi scambiati è fisso, LK adotta un **approccio adattivo**: il valore di k viene scelto dinamicamente in funzione del guadagno ottenuto durante la costruzione della sequenza di scambi. [20]

Algoritmo

L'algoritmo può essere descritto come segue, seguendo il modello originario presentato in [20] e le reinterpretazioni moderne di [3] [21] :

1. **Inizializzazione**: si parte da un tour iniziale completo, ad esempio costruito tramite un'euristica come Nearest Neighbor o Farthest Insertion [3].
2. **Scelta del nodo di partenza**: si seleziona un nodo iniziale t_1 del tour da cui iniziare la costruzione della sequenza di mosse.
3. **Costruzione della catena di scambi**: a partire da t_1 , si costruisce iterativamente una sequenza alternata di rimozioni e aggiunte di archi $(x_1, y_1), (x_2, y_2), \dots$, dove ogni x_i è un arco rimosso e ogni y_i è un nuovo arco inserito. La sequenza deve rispettare vincoli di ammissibilità (ovvero evitare la formazione di subtour) e mantenere la connessione del tour [3] [20] .
4. **Valutazione incrementale del guadagno**: dopo ogni coppia di mosse (x_i, y_i) si calcola il guadagno parziale come:

$$G_k = \sum_{i=1}^k (c(x_i) - c(y_i))$$

Se $G_k > 0$, la sequenza corrente viene salvata come candidata [20].

5. **Selezione e applicazione del miglior miglioramento:** una volta completata l'esplorazione delle sequenze (entro una profondità massima o secondo criteri euristici), si seleziona la sequenza candidata con guadagno massimo e la si applica al tour [3] [21]. Se nessuna sequenza migliora il tour, questo rimane invariato.
6. **Iterazione:** si ripete il processo scegliendo un nuovo nodo iniziale t_1 . Il procedimento termina quando non è più possibile ottenere alcun miglioramento da nessun nodo [20].

L'idea chiave dell'algoritmo è che la lunghezza della catena di scambi non sia prefissata, ma aumenti finché le mosse continuano ad essere **promettenti**, ovvero finché portano a miglioramenti incrementali della lunghezza del tour [3] [20].

Caratteristiche

- La lunghezza k della catena di scambi non è fissa, ma viene adattata dinamicamente in funzione del guadagno osservato.
- L'algoritmo può gestire riorganizzazioni complesse del tour, andando oltre le semplici mosse 2-opt o 3-opt.
- È particolarmente efficace su istanze **euclidee**, ovvero casi in cui i nodi sono punti nel piano e i costi tra coppie sono distanze euclidee simmetriche.
- In tali istanze, il tempo di esecuzione empirico osservato è circa $O(n^{2.2})$, rendendolo molto competitivo anche su grafi con migliaia di nodi [3].

Comportamento nel Caso Peggior

Sebbene l'algoritmo di *Lin–Kernighan* mostri un'elevata efficienza nella pratica e generi soluzioni di alta qualità su una vasta gamma di istanze, il suo comportamento teorico nel caso peggiore è meno favorevole.

In particolare, è stato dimostrato che l'algoritmo può richiedere un tempo esponenziale nel caso peggiore. Più precisamente, il numero di passi necessari può crescere fino a $O(n!)$ in alcune particolari configurazioni, dove n è il numero di città dell'istanza ovvero il numero di nodi. [3].

Tuttavia, nella pratica, l'algoritmo converge rapidamente e produce soluzioni vicine all'ottimo, rendendolo uno degli approcci euristici più efficaci per il TSP.

3.2.3 Lin–Kernighan–Helsgaun (LKH)

Descrizione

L'algoritmo *Lin–Kernighan–Helsgaun* (LKH), introdotto da Keld Helsgaun nei primi anni 2000 [21] [22], costituisce un'evoluzione dell'euristica di Lin–Kernighan, analizzata nella sezione appena presentata. L'obiettivo principale di LKH è raffinare ulteriormente il processo di miglioramento locale, rendendolo più efficiente e scalabile.

Algoritmo

Il funzionamento di LKH mantiene la struttura generale del metodo LK, ma ne espande significativamente l'efficacia attraverso accorgimenti tecnici ed euristici:

1. **Costruzione dei candidate sets:** per ogni nodo si seleziona un sottoinsieme di archi ritenuti promettenti, usando criteri come la distanza minima o il valore di α -nearness.
2. **Stima del potenziale di miglioramento:** si costruisce un 1-tree (un albero che collega tutti i nodi più un arco in più, ossia un **albero di copertura** con un arco in aggiunta.) per stimare l'impatto di ogni possibile mossa.
3. **Applicazione di mosse k -opt estese:** a differenza del LK classico, LKH ammette catene di scambi che non devono rispettare un ordine sequenziale, aumentando la varietà di tour esplorabili.
4. **Controllo della diversificazione:** se l'algoritmo converge verso una soluzione stabile senza miglioramenti, viene introdotta una lieve perturbazione per forzare l'esplorazione di nuove soluzioni.
5. **Strategia multi-start:** l'intera procedura viene ripetuta più volte con inizializzazioni diverse, e la miglior soluzione globale viene mantenuta.

Caratteristiche

- LKH è altamente efficiente su istanze simmetriche del TSP, arrivando a gestire problemi con oltre 100.000 nodi.
- Supporta estensioni del problema classico, come TSP asimmetrico, TSP con finestre temporali o costi vincolati [22].
- Combina velocità di esecuzione con grande accuratezza: in molti casi produce soluzioni vicine a quelle ottimali.
- L'utilizzo di perturbazioni intelligenti permette di simulare l'effetto di metaeuristiche, mantenendo però una struttura deterministica e veloce.

Comportamento nel Caso Peggior

Dal punto di vista teorico, LKH conserva la stessa complessità del LK originale: il numero di configurazioni da esplorare può crescere in modo esponenziale nel numero di nodi. Tuttavia, l'uso di strategie come i candidate sets e valutazioni euristiche consente di ridurre drasticamente lo spazio di ricerca. Nella pratica, il tempo medio di esecuzione si avvicina spesso a $O(n^2)$ su istanze ben strutturate [21].

3.3 Algoritmi esatti

3.3.1 Branch-and-Bound

Descrizione

Il metodo *Branch-and-Bound* per il TSP si è sviluppato parallelamente agli approcci basati sul Branch-and-Cut (descritti nella sezione successiva), guadagnando progressivamente popolarità a partire dagli anni '50. Le prime formulazioni parziali di tale metodo sono attribuite a Bock [25], Croes [26], Eastman [27], e Rossman & Twery [28], i cui algoritmi facevano uso di tecniche di enumerazione accompagnate da euristiche di **potatura**. Tuttavia, è con il lavoro di Little, Murty, Sweeney e Karel (1963) [29] che il termine "branch-and-bound" viene coniato e la metodologia sistematizzata per la prima volta.

Algoritmo

Il metodo branch-and-bound opera esplorando lo spazio delle soluzioni tramite un **albero di ricerca**, dove ogni nodo rappresenta un sottoproblema definito da vincoli aggiuntivi. Il metodo (come vedremo) combina due operazioni principali:

- **Branching**: suddivide un problema in sottoproblemi mutuamente esclusivi, ciascuno con vincoli aggiuntivi.
- **Bounding**: calcola un limite inferiore sul costo ottimo di ciascun sottoproblema, scartando quelli che non possono migliorare la soluzione corrente.

Il problema iniziale, definito come uno spazio di *soluzioni ammissibili* S , è posto nella forma:

$$\min\{c^T x \mid x \in S\}$$

e viene risolto **ricorsivamente** suddividendo lo spazio S in due sottoinsiemi disgiunti tramite vincoli lineari del tipo:

$$\alpha^T x \leq \beta' \quad \text{oppure} \quad \alpha^T x \geq \beta''$$

con $\beta' < \beta''$.

Ogni nodo dell'albero di ricerca ha quindi la forma:

$$\min\{c^T x \mid x \in S \text{ e } Cx \leq d\}$$

dove la matrice C ed il vettore d definiscono i vincoli specifici del sottoproblema.

Ad ogni apertura di un nuovo nodo, si calcola un limite inferiore (**bounding**) sul costo ottimo di tutti i sottoproblemi che tale nodo può generare. Tale limite, generalmente calcolato come soluzione di un problema lineare rilassato, viene usato come criterio di '**potatura**': quando risulta essere maggiore del costo del migliore tour noto durante l'esecuzione dell'algoritmo, il nodo viene 'potato', ossia non più considerato nel seguito dell'esplorazione, in quanto l'intero sottoramo dell'albero di ricerca ad esso associato non potrà portare soluzioni migliori di quella già trovata.

In caso contrario, si continua a generare sottoproblemi discendenti (**branching**).

L'efficacia del metodo sta proprio nella capacità di risolvere o escludere molti sottoproblemi senza mai doverci effettivamente entrare.

Branching frazionario

In presenza di soluzioni frazionarie (cioè quando x^* non è intero), il branching può essere implementato introducendo vincoli su una variabile x_e che ha valore non intero:

$$x_e \leq \lfloor x_e^* \rfloor \quad \text{oppure} \quad x_e \geq \lceil x_e^* \rceil$$

oppure, più in generale, scegliendo un vettore α intero tale che $\alpha^T x^*$ non sia intero, ponendo:

$$\alpha^T x \leq \lfloor \alpha^T x^* \rfloor, \quad \alpha^T x \geq \lceil \alpha^T x^* \rceil$$

Caratteristiche

- Il metodo garantisce la **correttezza** della soluzione: una volta completata l'esplorazione, il miglior tour trovato è ottimo.
- La qualità dell'approccio dipende fortemente dalla qualità dei limiti inferiori: rilassamenti più stretti permettono potature più aggressive.
- Può richiedere molto tempo e memoria per istanze di grandi dimensioni se non affiancato da strategie avanzate (es. branch-and-cut).

Comportamento nel Caso Peggior

Nel caso peggiore, il metodo esplora uno spazio **esponenziale** di sottoproblemi. Tuttavia, grazie all'operazione di potature nella pratica si osservano comportamenti migliori.

3.3.2 Branch-and-Cut

Descrizione

Il metodo *Branch-and-Cut* rappresenta un'evoluzione del classico approccio *Branch-and-Bound*, arricchendolo con l'introduzione dinamica di disuguaglianze valide (i cosiddetti *cutting planes*) durante la costruzione dell'albero di ricerca. L'idea alla base è semplice: combinare la precisione dei tagli, che rafforzano il modello lineare, con la flessibilità del branching, che consente di esplorare in modo mirato lo spazio delle soluzioni.

Algoritmo

L'algoritmo segue una logica iterativa: prima di suddividere il problema in sottoproblemi, si cerca di raffinarlo localmente aggiungendo vincoli che eliminano soluzioni non ammissibili.

1. **Risoluzione iniziale:** si risolve il rilassamento lineare del nodo corrente, ottenendo una soluzione x^* che può contenere valori frazionari.
2. **Separazione (FINDCUTS):** si verifica se x^* viola disuguaglianze valide note per il TSP, come le *subtour elimination constraints* o le *comb inequalities* [3] [30] .
3. **Aggiunta di tagli:** se vengono trovati tagli violati, questi vengono aggiunti al modello, e il rilassamento viene risolto nuovamente.
4. **Iterazione locale:** si ripete il ciclo di separazione finché non emergono nuovi tagli violati, oppure finché si raggiunge un limite prestabilito.
5. **Verifica di interezza:** se la soluzione è intera e ammissibile, può essere considerata candidata ottima.
6. **Branching:** se x^* è ancora frazionaria e non si possono aggiungere ulteriori tagli, si procede a suddividere il problema in due sottoproblemi più specifici.
7. **Potatura:** i rami vengono eliminati se il loro limite inferiore è peggiore della migliore soluzione intera trovata finora.

Questo processo viene applicato ricorsivamente nell'albero di ricerca.

Caratteristiche

- L'aggiunta sistematica di tagli migliora la precisione del rilassamento LP, avvicinando il suo valore a quello del problema intero.
- Il modulo FINDCUTS, se progettato con intelligenza, può essere riutilizzato in diversi nodi dell'albero, contribuendo a migliorare le prestazioni globali [3] [30] .
- Branch-and-Cut è il cuore di solutori come *Concorde* [3], uno dei più potenti al mondo per il TSP, capace di affrontare istanze con decine di migliaia di città.

Capitolo 4

Implementazione

4.1 Architettura del sistema

Il sistema `tsp-meta-solver` è stato progettato seguendo i principi di un'architettura software modulare, con una chiara separazione tra **frontend** e **backend**. Quest'ultimo è organizzato secondo un modello a tre livelli (*3-layer architecture*), suddiviso nei layer **Controller**, **Service** e **Repository**. Questa struttura è pensata per favorire l'estensibilità, la manutenzione del codice e l'indipendenza tra logica di business e gestione dei dati.

4.1.1 Struttura backend

1. Controller Layer: questo livello coordina l'esecuzione del sistema, è il mezzo di comunicazione tra frontend e backend. La logica nei controller è pensata per essere ad alto livello, delegando tutte le responsabilità computazionali al livello successivo (*Service*).

2. Service Layer: questo livello è interamente dedicato alla logica di business del progetto. Include l'implementazione dei vari solver TSP (Nearest Neighbor, Nearest Insertion, Farthest Insertion, LKH3, Concorde), i generatori di istanze difficili, e le strategie di esecuzione dei task.

3. Repository Layer: gestisce l'accesso, la rappresentazione e la configurazione dei dati. In particolare, si occupa della gestione delle impostazioni generali (sia del solver che per ogni algoritmo) e delle impostazioni di istanza, della creazione e scrittura dei report sperimentali e dei file rappresentanti le istanze difficili generate.

Motivazione della scelta architetturale

La scelta di adottare un'architettura a tre livelli è motivata da esigenze di:

- **Separazione delle responsabilità:** ogni layer è responsabile di un aspetto ben definito.

- **Facilità di manutenzione:** le modifiche in un layer non impattano gli altri, se non tramite interfacce pubbliche.
- **Estensibilità:** il sistema è progettato per essere facilmente estendibile con nuovi algoritmi, nuove strategie di esecuzione o nuovi formati di istanze, semplicemente aggiungendo nuove classi nei layer appropriati.
- **Testabilità:** ogni layer può essere testato in isolamento, facilitando lo sviluppo di test automatici.

In sintesi, questa architettura favorisce una gestione ordinata della complessità, facilitando al contempo lo sviluppo incrementale e l'adattamento del sistema a scenari futuri.

4.2 Interfacce

Affinché il sistema possa garantire l'estensibilità delle proprie funzionalità, molteplici **interfacce** sono messe a disposizione. Inoltre, all'interno dei vari layer si è preferito suddividere ulteriormente la varie funzionalità in ulteriori **moduli**.

Partendo dal livello **service**, i vari moduli con le rispettive interfacce:

- **Algorithm:**
 - *IAlgorithm*: definisce l'interfaccia comune per tutti gli algoritmi di risoluzione del TSP. Ogni algoritmo implementa il metodo **execute** e restituisce una soluzione.
 - *IPath*: rappresenta un cammino nel grafo, astratto da una sequenza di nodi e dal relativo costo.
 - *ISolution*: rappresenta una soluzione completa di una determinata istanza, includendo il cammino ottimale trovato.
 - *ISolutionCollector*: interfaccia responsabile della raccolta e gestione delle soluzioni generate da uno o più algoritmi. Consente di accumulare, filtrare o analizzare soluzioni.
- **Graph:**
 - *IGraph*: rappresenta un grafo, su cui si basa la formulazione del problema. Espone metodi per interrogare la struttura dati, indipendentemente dalla rappresentazione interna (matrice, lista di adiacenza, ecc.).
- **Problem:**
 - *IProblem*: definisce l'interfaccia per accedere a un'istanza del TSP, associando un grafo, un nome, e un insieme di metadati come dimensione e tipo di distanza.

- **Generator:**

- *IGenerator*: interfaccia per la generazione automatica di istanze difficili.
- *IShortestPath*: per l'implementazione di algoritmi di cammino minimo, come Floyd-Warshall o Dijkstra, utilizzati da alcuni solver o generatori.

- **Executor:**

- *IExecutor*: interfaccia che gestisce l'esecuzione degli algoritmi con le varie istanze caricate.
- *IExecutionTask*: rappresenta un task computazionale astratto, eseguibile da un executor. Ogni task incapsula un algoritmo, un'istanza e la relativa configurazione.

Osservando il livello **repository**, invece troviamo i seguenti moduli e interfacce:

- **Construction:**

- *InstanceConstruction*: interfaccia per costruire un'istanza del problema TSP (file .tsp) a partire da fonti diverse (file, configurazione, generatore).

- **Parser:**

- *IParser*: definisce l'interfaccia per lettura e interpretazione di formati specifici (es. JSON)

- **Reader:**

- *Reader*: interfaccia per la lettura dei file di input (istanze, .tsp)

- **Writer:**

- *Writer*: interfaccia responsabile della scrittura dei risultati computazionali.

- **Configuration:**

- *IConfigProvider*: interfaccia per l'accesso centralizzato a tutte le configurazioni del sistema, suddivise in globali e per istanza.
- *ISetting*: interfaccia base per tutte le configurazioni (sia globali che per istanza).
- *IInstanceSetting*: rappresenta le impostazioni specifiche per una singola esecuzione di un algoritmo su una determinata istanza.
- *IGeneralSetting*: rappresenta le impostazioni generali di un algoritmo, valide per tutte le esecuzioni indipendentemente dall'istanza.

4.3 Repository Layer

File di Configurazione `tspmetasolver.json`

Il sistema utilizza un file di configurazione in formato JSON, denominato `tspmetasolver.json`, che governa l'esecuzione dei vari algoritmi e i relativi parametri.

La struttura del file è organizzata in sezioni principali:

- **Execution:** definisce l'elenco degli algoritmi abilitati per l'esecuzione, ad esempio `NearestInsertion`, `NearestNeighbour`, `FarthestInsertion`, `Concorde` e `LKH3`.
- **Configurazioni per Algoritmo:** ciascun algoritmo è associato a una sezione dedicata, strutturata in due sottosezioni:
 - `GeneralSettings`, contenente i parametri globali validi per tutte le istanze;
 - `InstancesSettings`, contenente i parametri specifici per singole istanze.

Le euristiche `NearestInsertion`, `NearestNeighbour` e `FarthestInsertion` condividono una struttura parametrica simile. Tra i parametri comuni figurano:

- `SEED`, che imposta il seme per la generazione casuale, garantendo la ripetibilità degli esperimenti;
- `HardInstances`, che definisce un sottoinsieme di istanze considerate particolarmente sfidanti per l euristica (questo aspetto è approfondito in una sezione successiva).

Il significato di alcuni parametri dipende dall'algoritmo. Ad esempio:

- il parametro `Node`, per `NearestInsertion`, indica il numero di nodi dell'istanza generata;
- per `NearestNeighbour`, invece, `Node` si riferisce al parametro i usato nella costruzione del grafo, come descritto nel paper [1] (cfr. pagine 567–568).

L'algoritmo avanzato `LKH3` espone numerosi parametri di configurazione che permettono di controllare dettagliatamente il comportamento dell'algoritmo *Lin–Kernighan–Helsgaun*. Tra le impostazioni disponibili per ogni istanza si includono:

`PROBLEM_FILE`, `INITIAL_TOUR_FILE`, `INPUT_TOUR_FILE`, `MERGE_TOUR_FILE`,
`OUTPUT_TOUR_FILE`, `TOUR_FILE`

Anche `Concorde` supporta una serie di parametri specifici per il controllo fine dell'esecuzione, tra cui:

PROBLEM_FILE, OUTPUT_TOUR_FILE, INITIAL_TOUR_FILE, RESTART_FILE,
 EDGEGEN_FILE, INITIAL_EDGE_FILE, FULL_EDGE_FILE,
 EXTRA_CUT_FILE, CUTPOOL_FILE, PROBLEM_NAME

Il concetto di *Integrity Gap*, spesso utilizzato per valutare la difficoltà computazionale di un'istanza rispetto al rilassamento lineare del problema, sarà discusso nella sezione dedicata all'analisi delle **istanze difficili**.

Per ragioni di praticità, la configurazione relativa all'*Integrity Gap* è stata inclusa all'interno della sezione dedicata a Concorde, pur essendo un concetto teoricamente svincolato da questo algoritmo. Tale scelta è stata fatta al fine di mantenere una struttura omogenea con quella prevista per gli altri algoritmi nel file `tspmetasolver.json`.

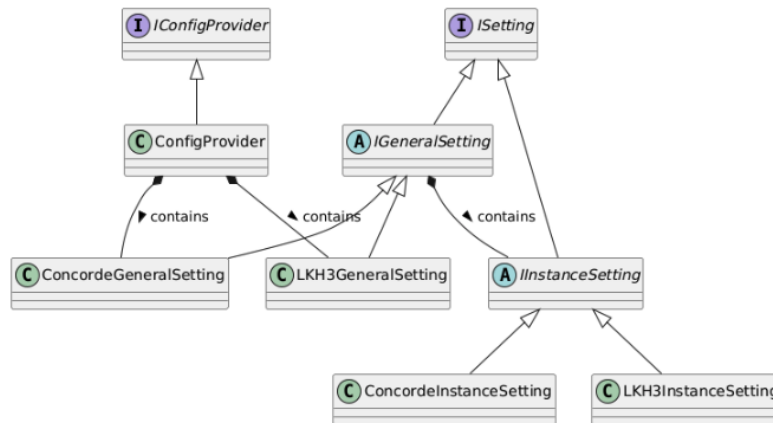


Figura 4.1: Diagramma UML delle classi e interfacce principali per la gestione delle configurazioni nel sistema. Vengono mostrati i livelli di astrazione tra le impostazioni generali e di istanza, nonché la relazione con il provider di configurazioni.

La figura 4.1 mostra la struttura delle classi e delle interfacce che modellano il sistema di configurazione. La lettura del file di configurazione è gestita dalla classe **ConfigProvider**, che si occupa di caricare e validare le informazioni presenti nel file di configurazione. Questa classe incapsula la logica di parsing del file e mette a disposizione i dati in una struttura adatta ai componenti del sistema. La **ConfigProvider** è a sua volta utilizzata dal **ConfigRepository**, che funge da punto di accesso centralizzato alla configurazione nel **Repository Layer**, fornendo un'interfaccia uniforme per accedere alle impostazioni relative ai diversi solver e delle singole istanze.

4.3.1 Gestione delle Istanze TSP

Attualmente, nel panorama accademico e industriale, un'istanza del problema del TSP può essere rappresentato in diversi formati, ciascuno con proprie caratteristiche. La **TSPLIB95**

[31] è una delle collezioni più note di benchmark, e definisce svariati formati per la rappresentazione delle distanze tra i vari nodi, tra cui quelli attualmente implementati all'interno del solver:

- **EUC_2D**: le coordinate dei nodi sono date in uno spazio euclideo bidimensionale. Le distanze sono calcolate con la distanza euclidea standard (arrotondata all'intero più vicino).
- **CEIL_2D**: simile a EUC_2D, ma la distanza euclidea viene arrotondata per eccesso anziché al più vicino.
- **GEO**: le coordinate rappresentano latitudine e longitudine. Le distanze sono calcolate sulla superficie terrestre assumendo una sfera terrestre di raggio 6378.388 km.
- **ATT**: distanza pseudo-euclidea, usata per modelli in cui la geometria non riflette perfettamente le distanze reali. Si basa su una formula approssimata e meno simmetrica.
- **EXPLICIT**: le distanze tra i nodi sono fornite esplicitamente tramite una matrice (completa o triangolare). È il formato più generale e flessibile, ma più ingombrante in termini di spazio.

4.3.2 Lettura delle Istanze TSP

La lettura delle istanze del problema del TSP è stata progettata per essere estendibile e modulare, tenendo conto delle diverse rappresentazioni di una data istanza, sfruttando il design pattern **Template Method** [33] per gestire i passi comuni e specifici della lettura, e il pattern **Chain of Responsibility** [33] per delegare il parsing al reader corretto.

L'interfaccia astratta **Reader** definisce il contratto per tutti i reader specifici, fornendo un metodo **read()** che incapsula la logica di lettura da file, che verrà poi sovrascritto dai rispettivi reader. L'attuale gerarchia può essere vista in figura 4.2.

La classe **TspReader**, derivata da **Reader**, implementa il metodo **read()** seguendo i principi del *Template Method* pattern: essa verifica in primis se l'attuale istanza del reader, se non è compatibile, passa il file al reader successivo, realizzando così una **catena di responsabilità** tra reader specializzati per formati diversi (es. ATT, CEIL_2D, GEO, ecc.).

Questo approccio consente di aggiungere facilmente **nuovi formati** rispetto a quelli già implementati, semplicemente estendendo **TspReader** e definendo il comportamento specifico nei metodi virtuali presenti.

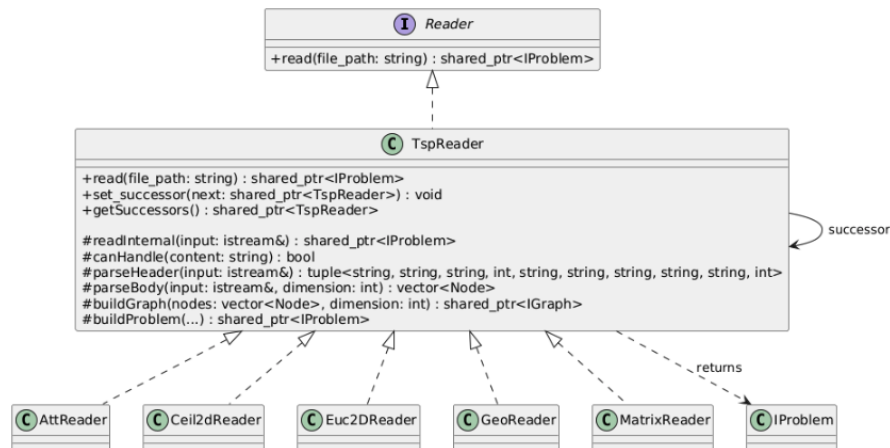


Figura 4.2: Gerarchia classi Reader

4.3.3 Creazione istanze TSP

Per valutare in modo efficace il comportamento degli algoritmi implementati, è stata introdotta una componente dedicata alla creazione di nuove istanze del problema. In particolare, si è voluto disporre di uno strumento in grado di generare in maniera controllata istanze *difficili*, ovvero configurazioni caratterizzate da proprietà che possono mettere in crisi specifiche strategie risolutive.

Per rispettare il principio di *separazione delle responsabilità* (**SOC**, *Separation of Concerns*, in inglese), il sistema è stato progettato in due moduli distinti: uno dedicato alla generazione dell'istanza sulla base di determinati parametri, e un altro responsabile della sua scrittura su file, nel formato standard `.tsp`.

La generazione avviene tramite la costruzione di una matrice di adiacenza (questo processo verrà descritto in seguito), la quale viene poi passata alla componente `InstanceConstruction`, un'interfaccia astratta pensata per supportare la creazione di rappresentazioni personalizzate in formati standardizzati. L'implementazione concreta `TspConstruction` si occupa della scrittura su file secondo le specifiche definite dalla **TSPLIB95** [31], producendo file con estensione `.tsp` compatibili con i più comuni risolutori.

4.3.4 Creazione dei report

Per fornire una rappresentazione persistente e leggibile dei risultati ottenuti dagli algoritmi di risoluzione, è stato progettato un sistema modulare per la scrittura dei report. In particolare, è stato adottato un approccio molto simile a quello proposto per la lettura delle istanze.

La classe astratta `Writer` definisce l'interfaccia principale per la scrittura, mentre la sua specializzazione `FileWriter` introduce una struttura *template* che separa la logica generica (invocazione, gestione del file, validazione del formato) dalla logica specifica di scrittura,

delegata a metodi virtuali da ridefinire. Questo approccio riflette il *Template Method* pattern, come si può vedere dalla figura 4.3.

Per gestire formati diversi (come .csv, .json, o altri formati futuri), il sistema adotta anche il pattern *Chain of Responsibility*: ogni `FileWriter` è dotato di un eventuale successore a cui può delegare l'operazione se non è in grado di gestire il formato richiesto. In questo modo, l'aggiunta di un nuovo formato richiede unicamente l'estensione della catena, senza alterare la logica delle componenti esistenti.

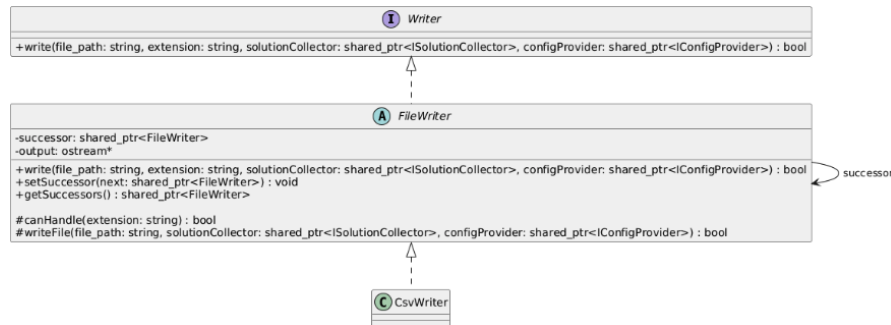


Figura 4.3: Gerarchia classi Writer

Tale design risulta particolarmente flessibile, poiché consente non solo di aggiungere nuovi formati di esportazione, ma anche di astrarre la destinazione dell'output. Ad esempio, sostituendo la scrittura su file con uno stream di output generico, sarebbe possibile visualizzare i risultati direttamente su console o inoltrarli a un'interfaccia grafica, mantenendo invariata l'interfaccia utente e la logica di business.

4.4 Service Layer

4.4.1 Generatore Istanze difficili

La generazione delle istanze difficili è un modulo centrale del sistema, progettato per testare l'efficacia degli algoritmi su input strutturati che presentano caratteristiche particolarmente sfidanti. Questo componente si basa su un'interfaccia generale, **IGenerator**, che definisce i metodi per la generazione di matrici di adiacenza che rappresentano grafi TSP.

Essendo queste istanze "particolari" cioè con collegamenti tra i nodi specifici al fine di rendere l'istanza stessa difficile per un risolutore specifico, la loro costruzione normalmente non prevede la creazione di un **grafo completo** - un grafo viene definito tale se esiste un arco per ogni coppia di vertici. Per tale motivo, tutti i generatori utilizzano un modulo interno per il calcolo dei cammini minimi tra i nodi che non hanno un collegamento diretto (**completamento metrico**), rappresentato dall'interfaccia **IShortestPath**. In particolare, in questa implementazione viene usato **FloydWarshall** [32].

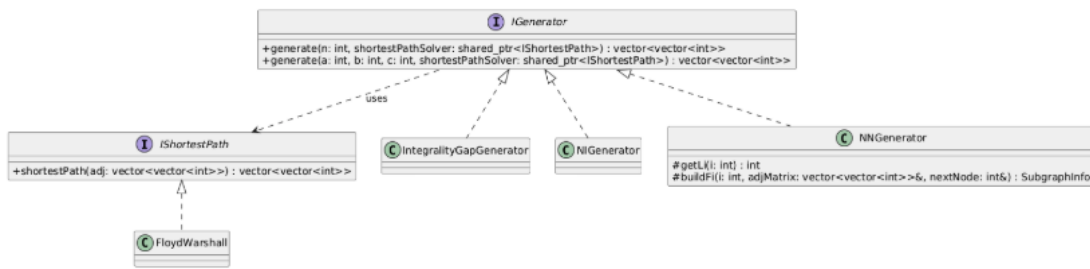


Figura 4.4: Gerarchia delle classi per la generazione di istanze difficili

Questa architettura, illustrata nella figura 4.4 permette di estendere facilmente il sistema con nuovi generatori o nuovi algoritmi di cammino minimo, sfruttando il principio dell'*inversione delle dipendenze*.

4.4.1.1 L'importanza delle istanze difficili

La generazione e l'analisi di istanze difficili rivestono un ruolo cruciale nella valutazione comparativa degli algoritmi per la risoluzione del TSP.

In letteratura, molte euristiche mostrano buone prestazioni su istanze casuali, ma falliscono in modo sistematico su configurazioni specifiche, appositamente progettate per innescare il loro comportamento nel caso peggiore. Queste istanze permettono di mettere in luce i limiti strutturali degli algoritmi e rappresentano un banco di prova fondamentale per la progettazione di tecniche più robuste e generalizzabili.

Le istanze difficili implementate nel nostro sistema si ispirano a costruzioni teoriche note, alle quali vengono applicate ε -**perturbazioni** mirate sui costi. Tali perturbazioni non sono casuali: esse sono studiate al fine di garantire che l'esecuzione dell'algoritmo ricada esattamente nel *caso peggiore* previsto dalla teoria, mantenendo al contempo la validità e la coerenza della configurazione.

È importante sottolineare che, nella letteratura, la presenza di istanze che inducono un comportamento worst-case per un algoritmo viene spesso dimostrata esistenzialmente, ovvero si prova che *esiste* almeno un'esecuzione di tale natura. Nel nostro lavoro, invece, l'obiettivo è far sì che l'algoritmo effettivamente *incappi* in queste esecuzioni peggiori, permettendo così una valutazione più precisa e affidabile delle prestazioni nel caso peggiore.

Nel seguito verranno descritte le istanze implementate per ciascuna famiglia di algoritmi (ad esempio Nearest Neighbour, Nearest Insertion, ecc.), specificando le fonti teoriche di riferimento, le modifiche introdotte e le caratteristiche che rendono ciascuna istanza particolarmente sfidante.

Questo approccio consente di svolgere un'analisi mirata e dettagliata, fornendo un contributo significativo alla comprensione e al miglioramento degli algoritmi studiati.

4.4.1.2 Nearest Neighbour

Come visto precedentemente, questa euristica risulta essere semplice e veloce. Tuttavia, può essere portata in condizioni di *worst-case* strutturando l'istanza in una determinata maniera, come mostrato da Rosenkrantz, Stearns e Lewis [1].

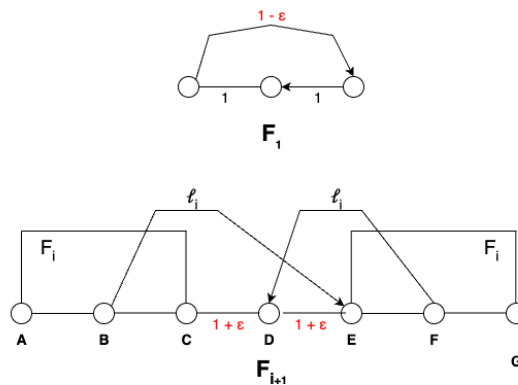


Figura 4.5: Esempio di istanza difficile per l'algoritmo *Nearest Neighbour*. L'immagine rappresenta una distribuzione dei nodi progettata per ingannare l'euristica greedy dell'algoritmo.

Fonte: adattata da [1].

L'intuizione alla base è quella di costruire un grafo dove il nodo iniziale (ad es. A) porta l'algoritmo a visitare successivamente il sottografo sbagliato, accumulando un costo molto maggiore rispetto all'ottimo.

Come anticipato nella Sezione 4.4.1.1, per garantire che l'euristica operi sempre nel caso peggiore, si applicano leggere ε -**perturbazioni** sugli archi, dove ε è scelto sufficientemente piccolo, per non alterare la metrica ma influenzare il comportamento della selezione locale.

Costruzione Ricorsiva delle Istanze Peggiori per Nearest Neighbour

Come mostrato originariamente da Rosenkrantz, Stearns e Lewis [1], la costruzione delle istanze difficili per l'euristica *Nearest Neighbour* si basa su una famiglia ricorsiva di sottografi F_i , ciascuno caratterizzato da tre nodi distintivi: **iniziale**, **centrale** e **finale**.

Definizione della struttura F_i

- Il grafo base F_1 è formato da tre nodi A, B, C connessi tra loro con archi di costo:

$$d(A, B) = 1, \quad d(B, C) = 1, \quad d(A, C) = 1 - \varepsilon,$$

I nodi A, B, C rappresentano rispettivamente il nodo **iniziale**, **centrale** e **finale** di F_1 .

- Per costruire F_{i+1} , si utilizzano due copie di F_i :

$F_i^{(1)}$ nodi iniziale, centrale e finale A, B, C , e $F_i^{(2)}$ con nodi E, F, G ,

e si introduce un nuovo nodo centrale D .

I nodi **iniziale**, **centrale** e **finale** di F_{i+1} sono quindi:

A, D, G .

- Gli archi aggiunti tra i nodi delle copie e D sono:

$$d(B, E) = \ell_i, \quad d(F, D) = \ell_i, \quad d(D, C) = 1 + \varepsilon, \quad d(D, F) = 1 + \varepsilon,$$

dove ℓ_i è definito ricorsivamente come:

$$\ell_i = \left\lfloor \frac{1 \cdot (4 \cdot 2^i - (-1)^i + 3)}{6} \right\rfloor.$$

Inoltre, gli archi interni delle copie $F_i^{(1)}$ e $F_i^{(2)}$ mantengono la struttura definita a livello i .

Osservazioni sui costi

- Gli archi *iniziale-centrale* e *centrale-finale* di ciascun sottografo F_i hanno costo 1.
- L'arco *iniziale-finale* in F_1 ha costo $1 - \varepsilon$.
- Le perturbazioni ε sono studiate per indirizzare l'euristica verso la scelta subottimale.

Lunghezza del percorso prodotto da NN

Il cammino selezionato dall'euristica NN segue inevitabilmente la concatenazione ingannevole dei sottografi F_i , ottenendo una lunghezza finale:

$$L_i = 6i \cdot 2^i + 8 \cdot 2^i + (-1)^i - 9 + \delta_\varepsilon,$$

dove δ_ε rappresenta una piccola correzione dovuta alle perturbazioni ε .

La lunghezza del percorso ottimo, invece, è significativamente inferiore, garantendo un rapporto di approssimazione:

$$\frac{L_{\text{NN}}}{L_{\text{OPT}}} \sim \log n,$$

dove $n = |V(F_i)|$ è il numero di nodi in F_i , che cresce esponenzialmente con i .

Dimensione dell'istanza in funzione della ricorsione

Il generatore **NNGenerator** riceve come parametro il numero di ricorsioni i e costruisce ricorsivamente la matrice di adiacenza:

$$|V(F_i)| = 2^{i+1} - 1,$$

ad esempio:

- $i = 1$ produce un'istanza con 3 nodi;
- $i = 2$ produce un'istanza con 7 nodi;
- $i = 3$ produce un'istanza con 15 nodi;
- e così via.

La funzione `buildFi()` gestisce la costruzione ricorsiva di F_i , aggiungendo i nodi e gli archi con i costi sopra descritti. La matrice risultante viene quindi completata metricamente tramite il modulo **IShortestPath**, utilizzando l'algoritmo di Floyd–Warshall per garantire la completezza metrica [32].

4.4.1.3 Nearest Insertion

Per la generazione delle istanze difficili per l'euristica *Nearest Insertion* si adotta un approccio simile a quello utilizzato per *Nearest Neighbour*.

Sebbene questa euristica risulti efficace su molte istanze casuali, è noto che può essere indotta a comportarsi nel suo *worst-case*, come dimostrato da Rosenkrantz et al. [1]. Le istanze difficili sono progettate in modo tale che l'inserimento del nodo “più vicino” porti l'algoritmo a costruire un percorso subottimale significativamente più lungo dell'ottimo.

Costruzione delle Istanze Peggiori per Nearest Insertion

La costruzione prevede un grafo con:

- n nodi disposti ai vertici di un poligono regolare;
- archi lungo il perimetro del poligono con costo esattamente 1;
- archi “intermedi” (chiamati anche “corde”) con costo $2 - \varepsilon$.

Queste ε -perturbazioni sono scelte in modo mirato per forzare l'euristica ad effettuare inserimenti “subottimali” nell'ordine previsto, inducendo un percorso finale più lungo.

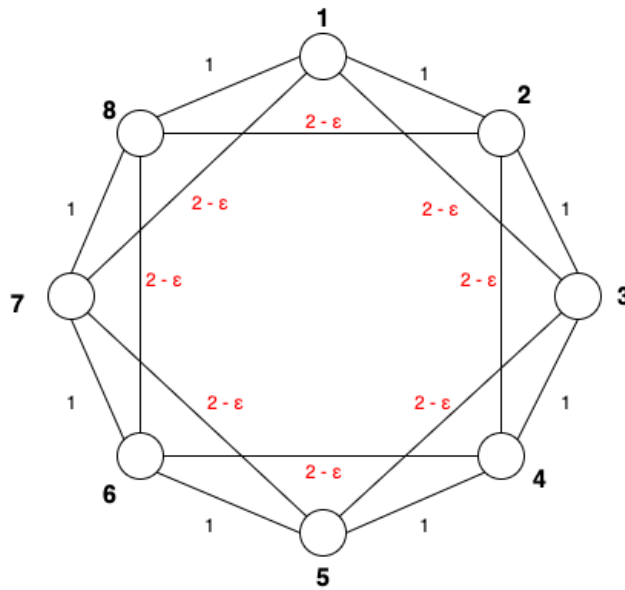


Figura 4.6: Esempio di istanza difficile per l'euristica *Nearest Insertion*. I nodi sono disposti ai vertici di un poligono regolare con archi lungo il perimetro di costo 1 e archi "intermedi" leggermente meno costosi.

Fonte: adattata da [1].

Lunghezza del percorso prodotto da NI

In particolare, la lunghezza del tour prodotto dall'euristica viene calcolata come:

$$L(n) = 2(n - 1) - (n - 2)\varepsilon,$$

che si avvicina a $2(n - 1)$ per $\varepsilon \rightarrow 0$.

Il percorso ottimo, invece, è esattamente il ciclo che attraversa tutti i nodi nell'ordine del poligono, con costo:

$$OPT(n) = n,$$

poiché:

- ogni tour ha esattamente n archi;
- ogni arco ha costo almeno 1;
- il ciclo perimetrale ha costo esattamente n , ed è quindi ottimo.

Da ciò deriva che il rapporto di approssimazione si avvicina a 2, che corrisponde al *worst-case bound* teorico dell'algoritmo [1].

La costruzione concreta è implementata dalla classe **NIGenerator**, che genera la matrice di adiacenza posizionando i nodi secondo lo schema descritto. Per rendere il grafo com-

pleto e metrico, viene applicato un modulo di completamento metrico basato sull'interfaccia `IShortestPath`, che calcola i cammini minimi tra nodi non direttamente connessi (ad esempio con l'algoritmo di Floyd–Warshall) [32].

L'input del generatore è il numero di nodi n , corrispondente alla dimensione dell'istanza prodotta.

4.4.1.4 Integrality Gap

L'**Integrality Gap** (IG) è una misura che quantifica la distanza tra la soluzione ottima del problema intero e quella di un suo rilassamento lineare (ad esempio, la programmazione lineare sul problema del TSP). E' importante evidenziare che per un dato problema intero possono esserci diversi rilassamenti lineari, e, diversi rilassamenti hanno diversi IG. Il valore di IG dipende dal tipo di rilassamento LP adottato e fornisce un'indicazione teorica sulla "difficoltà" strutturale del problema.

Le istanze con IG elevato non sono necessariamente difficili da risolvere per gli algoritmi euristici analizzati in questa tesi, ma, data la loro complessità teorica intrinseca, rappresentano un interessante banco di prova complementare.

Benoit e Boyd [41] introducono una famiglia di istanze, denotate $F(a, b, c)$, costruite per stressare l' *integrality gap* della subtour LP (Held–Karp). La struttura di $F(a, b, c)$ prevede tre catene lineari di nodi:

Catena A : $A_0, A_1, \dots, A_a$, Catena B : $B_0, B_1, \dots, B_b$, Catena C : $C_0, C_1, \dots, C_c$,

dove le catene contengono rispettivamente $a+1$, $b+1$ e $c+1$ nodi, per un totale di $a+b+c+3$ nodi. Poiché il valore esatto del massimo gap di integrality non è noto, non si può affermare che le istanze $F(a, b, c)$ lo massimizzino in senso assoluto; tuttavia, per scelte opportune di (a, b, c) , esse conseguono i più alti valori di gap riportati finora in letteratura.

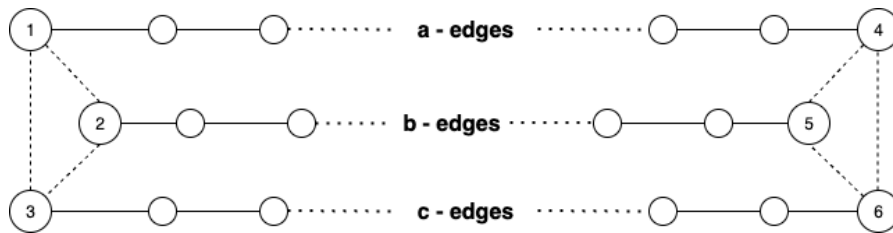


Figura 4.7: Esempio costruzione dell'istanza $F(a, b, c)$ per il calcolo dell'Integrality Gap.

Fonte: adattata da [2].

Gli archi all'interno delle catene hanno pesi costanti:

- Per ogni $i = 0, \dots, a - 1$:

$$d(A_i, A_{i+1}) = b \cdot c,$$

- Per ogni $i = 0, \dots, b - 1$:

$$d(B_i, B_{i+1}) = a \cdot c,$$

- Per ogni $i = 0, \dots, c - 1$:

$$d(C_i, C_{i+1}) = a \cdot b.$$

Le estremità delle catene sono collegate tra loro secondo lo schema:

- Archi tra i primi nodi delle catene:

$$d(A_0, B_0) = a \cdot c + b \cdot c,$$

$$d(A_0, C_0) = a \cdot b + b \cdot c,$$

$$d(B_0, C_0) = a \cdot b + a \cdot c,$$

- Archi tra gli ultimi nodi delle catene:

$$d(A_a, B_b) = a \cdot c + b \cdot c,$$

$$d(A_a, C_c) = a \cdot b + b \cdot c,$$

$$d(B_b, C_c) = a \cdot b + a \cdot c.$$

Il generatore **IntegrityGapGenerator** riceve in input la tripla (a, b, c) e costruisce la matrice di adiacenza corrispondente a questa struttura. La matrice generata non è inizialmente completa, pertanto per garantire la proprietà metrica viene applicato un **completamento metrico** utilizzando un modulo interno basato sull'interfaccia `IShortestPath`.

4.4.1.5 Concorde

Nel panorama accademico sono già disponibili diverse raccolte di istanze progettate per mettere in difficoltà il solver esatto **Concorde**. In particolare, si è deciso di utilizzare le istanze pubblicamente disponibili. Le principali fonti utilizzate sono:

- Le istanze proposte da Hougardy e Zhong [34], progettate appositamente per essere difficili da risolvere anche con algoritmi esatti ¹.
- Le istanze della **HardTSPLIB** generate da Vercesi et al. [35], progettate per presentare un elevato integrity gap nel rilassamento lineare del TSP ²

¹<http://webhotel4.ruc.dk/~keld/research/LKH/>

²<https://github.com/eleonoravercesi/HardTSPLIB>

Qualora si rendesse necessario sviluppare generatori personalizzati per questo tipo di istanze, è sufficiente seguire il modello utilizzato per gli altri generatori, estendendo l'interfaccia `IGenerator` con la logica di costruzione desiderata.

4.4.2 Implementazione degli algoritmi

Il modulo dedicato agli algoritmi è stato implementato seguendo un approccio modulare e flessibile, in cui ogni algoritmo concretizza l'interfaccia `IAlgorithm`, facilitando l'integrazione di nuovi risolutori e la manutenzione di quelli esistenti. Questa interfaccia definisce un metodo principale:

```
1 virtual std::shared_ptr<ISolution> execute(std::shared_ptr<
    IProblem> problem, std::shared_ptr<IInstanceSetting>
    instanceSettings) = 0;
```

Listing 4.1: Metodo principale dell'interfaccia `IAlgorithm`

Il metodo `execute` prende in input un'istanza del problema (che implementa `IProblem`) e una configurazione specifica dell'algoritmo (derivata da `IInstanceSetting`), restituendone la soluzione (`ISolution`).

Le euristiche implementato fino a questo momento sono quelle analizzate fin'ora:

- `NearestNeighbourSolver`
- `NearestInsertionSolver`
- `FarthestInsertionSolver`
- `ConcordeSolver` (che invoca il solver esterno Concorde)
- `LKH3Solver` (che invoca il solver esterno LKH3)

Ognuno di questi algoritmi ha un proprio oggetto di configurazione, derivato da `IInstanceSetting`, in cui sono specificati i parametri necessari per l'esecuzione dell'istanza data in input con il risolutore utilizzato. Nella figura 4.8 viene illustrata la struttura ad alto livello che governa l'interazione tra gli algoritmi, i problemi e le impostazioni di esecuzione.

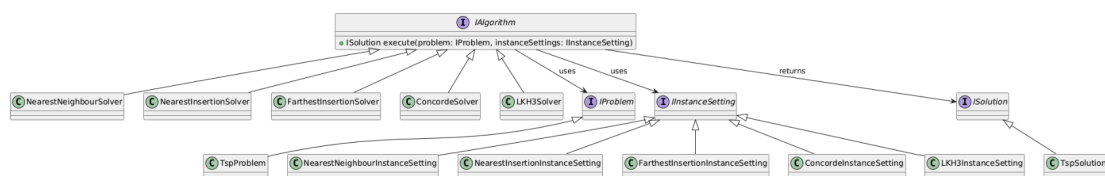


Figura 4.8: Diagramma UML che mostra la gerarchia e le relazioni tra l'interfaccia `IAlgorithm`, i solver implementati e le rispettive configurazioni.

4.4.3 Sistema di esecuzione

Il sistema di esecuzione ha il compito di gestire in modo flessibile e scalabile l'avvio, il monitoraggio e la raccolta delle soluzioni prodotte dagli algoritmi applicati ai problemi specificati. Nella figura 4.9 viene illustrata la struttura ad alto livello di questo modulo. Questa presenta una facile estensione del sistema, rendendo possibile l'integrazione di nuove strategie di esecuzione (ad esempio parallela o distribuita) e mantenendo al contempo una netta separazione tra i diversi compiti: definizione dell'algoritmo, configurazione dell'istanza, esecuzione e raccolta dei risultati.

Il ruolo centrale in questo sistema è svolto dall'interfaccia **IExecutor**, che definisce il contratto per l'aggiunta di task di esecuzione, l'avvio dell'elaborazione e l'accesso ai risultati. In questo progetto si è scelto di adottare una strategia di esecuzione sequenziale, concretizzata attraverso la classe **SingleQueueExecutor**.

Fondamentali in questa architettura sono anche le **ExecutionTask** (implementazioni di *IExecutionTask*), che rappresentano unità autonome di lavoro: ognuna incapsula l'esecuzione di un singolo algoritmo su una determinata istanza del problema con una specifica configurazione. Questo approccio permette di scomporre il carico computazionale in task indipendenti, rendendo semplice in futuro l'estensione verso modalità di esecuzione concorrente o distribuita.

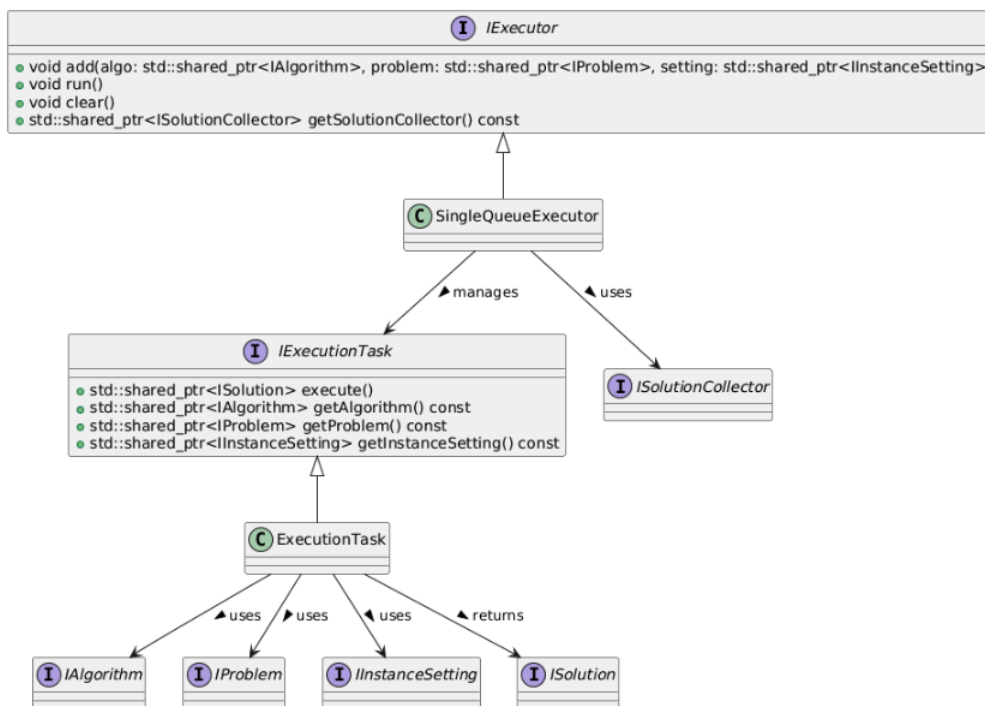


Figura 4.9: Diagramma UML del sistema di esecuzione, con le principali classi e interfacce coinvolte nell'orchestrazione degli algoritmi.

4.5 Controller Layer

Come anticipato, questo livello rappresenta il punto di comunicazione tra il *frontend* e il *backend*. Ogni richiesta viene gestita da un controller dedicato, il cui compito principale è orchestrare il flusso di dati verso il livello di servizio sottostante, delegando la logica applicativa vera e propria.

La scelta progettuale di avere un controller distinto per ciascuna funzionalità nasce dalla volontà di garantire una netta separazione delle responsabilità, in linea con i principi SOLID. Questa struttura migliora la manutenibilità e consente di estendere il sistema in modo modulare.

Tutti i controller presentano una struttura architetturale coerente, illustrata genericamente nella Figura 5.8, dove ciascun controller comunica con uno o più service, i quali a loro volta possono fare uso di componenti come repository, generatori o moduli esterni.

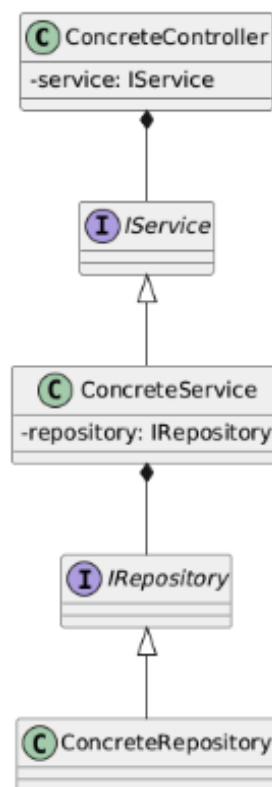


Figura 4.10: Diagramma generico che riassume la relazione tra i tre layer: Controller, Service, Repository.

Nel sistema sono stati implementati quattro controller principali, ciascuno con responsabilità ben definite:

- **TspController**: interfaccia per il caricamento di istanze del problema e la generazione dei report finali.
- **ExecutorController**: gestisce l'aggiunta, l'esecuzione e la pulizia dei task di calcolo.
- **ConfigController**: si occupa del caricamento e della validazione delle configurazioni generali e specifiche per algoritmo o istanza.
- **ConstructionController**: consente la costruzione dinamica di istanze difficili tramite diversi generatori (NI, FI, NN, IG).

Tutti i controller sono progettati come `singleton`, al fine di garantire una sola istanza condivisa all'interno del sistema, ed espongono metodi pubblici che astraggono completamente la logica sottostante, il tutto permette di avere un'interfaccia semplice per l'utente.

4.6 Inizializzazione del Sistema

Il diagramma in figura 4.11 mostra il processo di inizializzazione del sistema effettuato dal metodo `Initializer::init`. Le fasi principali del processo sono:

- Lettura e parsing del file di configurazione (*tspmetasolver.json*).
- Generazione automatica delle istanze difficili secondo i parametri specificati.
- Lettura delle istanze TSP fornite dall'utente.
- Creazione e configurazione dinamica degli algoritmi abilitati.
- Preparazione ed esecuzione dei task algoritmici.
- Raccolta e salvataggio dei risultati finali in formato CSV.

Questo flusso assicura modularità, estendibilità e una chiara separazione delle responsabilità tra i componenti, in linea con i principi dell'architettura a tre livelli adottata nel progetto.

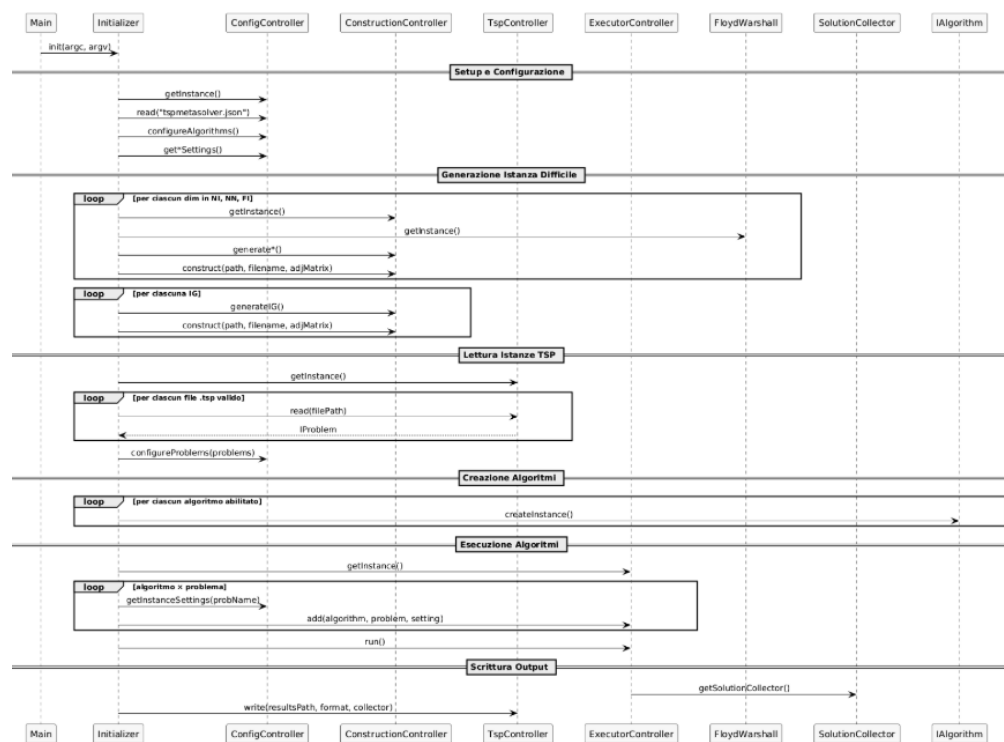


Figura 4.11: Diagramma di sequenza del metodo `Initializer::init`

Capitolo 5

Risultati Sperimentali

5.1 Algoritmi Testati

Gli algoritmi testati includono:

- **Euristiche costruttive:** Nearest Neighbor (NN), Nearest Insertion (NI), Farthest Insertion (FI)
- **Algoritmi di miglioramento:** Lin-Kernighan-Helsgaun (LKH3)
- **Solver esatti:** Concorde

Tutti gli algoritmi sono stati integrati in un sistema software modulare, descritto nel Capitolo 4.

5.2 Set di Istanze Utilizzato

5.2.1 Istanze Standard

Le istanze standard sono state selezionate dalla libreria TSPLIB95¹. Le principali caratteristiche delle istanze selezionate sono riportate in Tabella 5.1.

Tabella 5.1: Istanze simmetriche TSP da *TSPLIB95*

Istanza	Nodi
a280	280
ali535	535
att48	48
att532	532

Continua nella pagina successiva

¹<http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp/>

Tabella 5.1 – continua dalla pagina precedente

Istanza	Nodi
bayg29	29
bays29	29
berlin52	52
bier127	127
brg180	180
ch130	130
ch150	150
d198	198
d493	493
d657	657
d1291	1291
d1655	1655
d2103	2103
d15112	15112
eil51	51
eil76	76
eil101	101
fl417	417
gr17	17
gr21	21
gr24	24
gr48	48
gr96	96
gr120	120
gr202	202
gr229	229
gr431	431
gr666	666
hk48	48
kroA100	100
kroB100	100
kroC100	100
kroD100	100
kroE100	100
kroA150	150
kroB150	150

Continua nella pagina successiva

Tabella 5.1 – continua dalla pagina precedente

Istanza	Nodi
kroA200	200
kroB200	200
lin105	105
lin318	318
pcb1173	1173
pcb3038	3038
pr76	76
pr107	107
pr124	124
pr136	136
pr144	144
pr152	152
rat99	99
rat195	195
rat575	575
rat783	783
rd100	100
rd400	400
st70	70
ts225	225
tsp225	225
u159	159
u574	574
u724	724
ulysses16	16
ulysses22	22
vm1084	1084
vm1748	1748

5.2.2 Generazione delle Istanze Difficili

Le istanze difficili sono state generate seguendo le specifiche presentate nel Capitolo 4, con lo scopo di mettere in crisi particolari euristiche e analizzare il loro comportamento nei casi peggiori.

- **Istanze per Nearest Neighbor (NN):** costruite secondo la metodologia proposta in [1], variando il numero totale di nodi n dell'istanza.

Tabella 5.2: Istanze generate per il caso peggiore di Nearest Neighbor (*HARDNN*)

Parametro n	Nodi generati
5	63
10	2047
11	4095
12	8191

- **Istanze per Nearest Insertion (NI):** generate seguendo la costruzione presentata in [1].

Tabella 5.3: Istanze generate per il caso peggiore di Nearest Insertion (*HARDNI*)

Parametro n	Nodi generati
8	8
15	15
30	30
50	50
100	100
200	200
500	500
1000	1000
2000	2000
2500	2500
3000	3000
4000	4000
5000	5000

- **Istanze per il gap di integrality:** basate sulla costruzione di [41], caratterizzate da parametri regolabili (a, b, c).

Tabella 5.4: Istanze basate sulla costruzione di Benoit e Boyd per massimizzare l'integrality gap (*HIGH_IG*)

a	b	c	Nodi generati
3	2	3	11

Continua nella pagina successiva

Tabella 5.4 – continua dalla pagina precedente

<i>a</i>	<i>b</i>	<i>c</i>	Nodi generati
4	5	6	18
5	5	5	18
6	6	6	21
6	8	9	26
7	7	7	24
8	8	8	27
9	9	9	30
10	10	10	33
12	15	17	47
15	15	15	48
20	20	20	63
25	30	35	93
45	50	55	153
90	100	110	303
180	200	220	603
450	500	550	1503

5.2.3 Altre Istanze Difficili

Oltre alle istanze presenti in TSPLIB95 e alle costruzioni note in letteratura, sono stati utilizzati anche altri tipi di istanze difficili, recentemente proposti per testare i limiti di alcuni risolutori:

- **Vercesi, Gualandi, Mastrolilli, Gambardella (2023):** sono state considerate le istanze presentate in [35], raccolte nella libreria *HardTSPLIB*², in cui vengono generati problemi TSP metrici con ampio *integrality gap*, particolarmente difficili da risolvere con metodi esatti come il *branch-and-cut*.

Tabella 5.5: Istanze libreria *HARDTSPLIB*

Istanza	Nodi
10001_hard	10
10007_hard	10
10008_hard	10
10010_hard	10
11675_hard	11

Continua nella pagina successiva

²<https://github.com/eleonoravercesi/HardTSPLIB>

Tabella 5.5 – continua dalla pagina precedente

Istanza	Nodi
12290_hard	12
14850_hard	14
15002_hard	15
15005_hard	15
15007_hard	15
16038_hard	16
20004_hard	20
20007_hard	20
20009_hard	20
20181_hard	20
25001_hard	25
25004_hard	25
25006_hard	25
30001_hard	30
30003_hard	30
30005_hard	30
33001_hard	33
35002_hard	35
35003_hard	35
35009_hard	35
40003_hard	40
40004_hard	40
40008_hard	40
att48_hard	48
bayg29_hard	29
bays29_hard	29
brazil58_hard	58
dantzig42_hard	42
eil51_hard	51
gr24_hard	24
gr48_hard	48
hk48_hard	48
pr76_hard	76
st70_hard	70
swiss42_hard	42

- **Istanze generate da Helsgaun:** alcune istanze artificiali, ideate originariamente da

Hougardy e Zhong [34] e pubblicate nella pagina ufficiale di Keld Helsgaun³, sono state prese in considerazione. Queste istanze, spesso utilizzate come benchmark avanzati per LKH e altri algoritmi euristici, presentano elevata difficoltà di risoluzione; per alcune di esse è nota la soluzione ottima.

Tabella 5.6: Istanze della famiglia *Tnm*

Istanza	Nodi
Tnm52	52
Tnm55	55
Tnm58	58
Tnm61	61
Tnm64	64
Tnm67	67
Tnm70	70
Tnm73	73
Tnm76	76
Tnm79	79
Tnm82	82
Tnm85	85
Tnm88	88
Tnm91	91
Tnm94	94
Tnm97	97
Tnm100	100
Tnm103	103
Tnm106	106
Tnm109	109
Tnm112	112
Tnm115	115
Tnm118	118
Tnm121	121
Tnm124	124
Tnm127	127
Tnm130	130
Tnm133	133
Tnm136	136
Tnm139	139

Continua nella pagina successiva

³<http://webhotel4.ruc.dk/~keld/research/LKH/>

Tabella 5.6 – continua dalla pagina precedente

Istanza	Nodi
Tnm142	142
Tnm145	145
Tnm148	148
Tnm151	151
Tnm154	154
Tnm157	157
Tnm160	160
Tnm163	163
Tnm166	166
Tnm169	169
Tnm172	172
Tnm175	175
Tnm178	178
Tnm181	181
Tnm184	184
Tnm187	187
Tnm190	190
Tnm193	193
Tnm196	196
Tnm199	199

5.3 Macchina utilizzata

Gli esperimenti sono stati condotti su un'istanza **Amazon EC2 r5.2xlarge**, selezionata per l'elevata quantità di memoria e le buone prestazioni in ambienti multi-thread. Le caratteristiche principali della macchina sono riportate di seguito:

- **Tipo istanza:** r5.2xlarge (AWS EC2)
- **vCPU:** 8 (Intel Xeon Platinum 8000 series, architettura x86₆₄) **RAM:** 64GBDDR4
- **Storage:** SSD EBS (Elastic Block Store)
- **Sistema operativo:** Ubuntu Server 24.04 LTS (64 bit)
- **Compilatore C++:** GCC 13.3.0
- **Ambiente di build:** CMake 3.28.3 + Ninja 1.11.1

5.4 Configurazione degli esperimenti

Ogni algoritmo è stato eseguito **10 volte** su ciascuna istanza, utilizzando **10 semi diversi** per generare la casualità nei processi. All'interno di ciascuna esecuzione, tutti gli algoritmi sono stati valutati con lo **stesso seed**, per garantire condizioni di partenza comparabili. L'algoritmo **Concorde** è stato eseguito con le impostazioni di default, come da distribuzione ufficiale.

Per quanto riguarda **LKH3**, sono stati utilizzati i seguenti parametri di configurazione:

- MAX_TRIALS = 1000
- RUNS = 1
- TRACE_LEVEL = 1
- MAX_CANDIDATES = 5
- PATCHING_C = 3
- PATCHING_A = 2
- MOVE_TYPE = 5
- BACKBONE_TRIALS = 1000
- GAIN23 = YES
- SUBGRADIENT = NO
- RESTRICTED_SEARCH = YES

Al termine delle esecuzioni, i risultati sono stati aggregati in un file `.csv`, in cui per ciascuna istanza è stato calcolato quanto segue:

- **Tempo medio di esecuzione (Time_Mean)**
- **Varianza del tempo (Time_Var)**
- **Rapporto tra la soluzione trovata e l'ottimo (Ratio = ALGO/OPT)**
- **Media del rapporto (Ratio_Mean)**
- **Varianza del rapporto (Ratio_Var)**

5.5 Visualizzazione dei risultati

Per analizzare i risultati delle esecuzioni, sono stati generati diversi grafici che evidenziano l'andamento delle prestazioni dei vari algoritmi sia in termini di accuratezza (rapporto ALG/OPT) sia di tempo di esecuzione. I grafici sono stati realizzati utilizzando `matplotlib`, `seaborn` e `pandas`, e sono organizzati secondo diversi criteri di aggregazione.

5.5.1 Premessa su ALG/OPT e OPT

Nella sezione sperimentale, i rapporti ALG/OPT vengono calcolati come il rapporto tra il costo della soluzione ottenuta dall'algoritmo e il valore ottimale di riferimento (OPT).

Nota importante: il valore OPT può avere due significati a seconda della classe di istanze:

- *OPT certificato*: valore ottimale noto e garantito, derivante da fonti ufficiali. Le classi TNM e TSPLIB rientrano in questa categoria.
- *OPT empirico*: il miglior valore trovato tra tutte le esecuzioni dei solver testati. Questo approccio è utilizzato per le classi di istanze, dove un OPT certificato non è disponibile. In particolare, le classi HARDNI, HARDNN e HARDTSPLIB rientrano in questa categoria.

Questa distinzione è fondamentale per interpretare correttamente i rapporti ALG/OPT: nei casi in cui OPT è empirico, alcuni algoritmi possono avere ratio molto prossimi a 1, perché il valore di riferimento coincide con la loro migliore esecuzione.

5.5.2 Grafici per algoritmo e classe

Per ciascun algoritmo e per ciascuna classe di istanze, è stato generato un grafico combinato composto da due sottografici:

- **Rapporto ALG/OPT (media $\pm$ varianza)**: rappresenta l'accuratezza dell'algoritmo rispetto alla soluzione ottima, con barre di errore che indicano la varianza sulle dieci esecuzioni.
- **Tempo di esecuzione medio (media $\pm$ varianza)**: mostra il tempo impiegato dall'algoritmo per risolvere l'istanza, sempre con barre di errore.

L'asse delle ordinate è adattato dinamicamente tramite una funzione che analizza la variabilità dei dati e determina i limiti di visualizzazione più significativi. Questo approccio garantisce una lettura più efficace, soprattutto nei casi in cui i rapporti ALG/OPT sono molto vicini a 1.

5.5.3 Grafici aggregati per fascia dimensionale

Le istanze sono state suddivise in fasce dimensionali, come indicato di seguito:

Fascia	Dimensione (nodi)
1	$1 \leq n \leq 50$
2	$51 \leq n \leq 100$
3	$101 \leq n \leq 200$
4	$201 \leq n \leq 1000$
5	$1001 \leq n \leq 2000$
6	$2001 \leq n \leq 5000$
7	$n > 5000$

Per ciascuna fascia, sono stati prodotti due grafici a barre:

- **Media del rapporto ALG/OPT:** aggregata per algoritmo e suddivisa per classe. È presente una linea tratteggiata a quota 1.0 per identificare la soluzione ottima.
- **Tempo medio di esecuzione:** su scala logaritmica, utile per confrontare algoritmi con ordini di grandezza molto diversi.

5.5.4 Grafici combinati per classe

Per ogni classe di istanze, sono stati tracciati i seguenti grafici:

- **Tempo medio di esecuzione:** confronto tra algoritmi su diverse dimensioni.
- **Media ALG/OPT:** confronto della qualità della soluzione tra algoritmi.

Questi grafici consentono di valutare, per ciascuna classe, quali algoritmi risultano più efficienti e/o precisi al variare della dimensione del problema.

5.5.5 Grafici combinati per algoritmo

Infine, per ciascun algoritmo sono stati generati grafici che mostrano l'andamento su tutte le classi:

- **Tempo di esecuzione:** per classe e dimensione.
- **Accuratezza (ALG/OPT):** per classe e dimensione.

In questi plot si osservano le eventuali sensibilità dell'algoritmo a diverse tipologie di istanza e la sua scalabilità con l'aumentare del numero di nodi.

5.5.6 Note aggiuntive

La conversione del tempo è stata effettuata da microsecondi a millisecondi per una rappresentazione più intuitiva. Inoltre, è stata applicata una scala logaritmica all'asse dei tempi nei grafici aggregati per evitare che i tempi più elevati rendessero poco visibili quelli più piccoli.

5.6 Analisi dei risultati

5.6.1 Grafici aggregati per fascia dimensionale

5.6.1.1 Dimensioni ridotte (0-50 nodi)

Nelle dimensioni più contenute, si osserva una sostanziale equivalenza tra gli algoritmi in termini di qualità della soluzione, con rapporti ALG/OPT molto prossimi a 1.0 per tutti gli approcci testati, a eccezione del Nearest Insertion che performa "male" sulle sue istanze difficili. Tuttavia, emergono significative differenze nei tempi di esecuzione: Concorde, LKH3, FarthestInsertion, NearestInsertion e NearestNeighbour mostrano comportamenti molto diversi: mentre Concorde, FarthestInsertion, NearestInsertion e NearestNeighbour mantengono tempi dell'ordine di 10-30 ms, LKH3 mostra già tempi considerevolmente superiori ($\approx 10^4$ ms), suggerendo una complessità computazionale intrinseca elevata anche per istanze di piccole dimensioni.

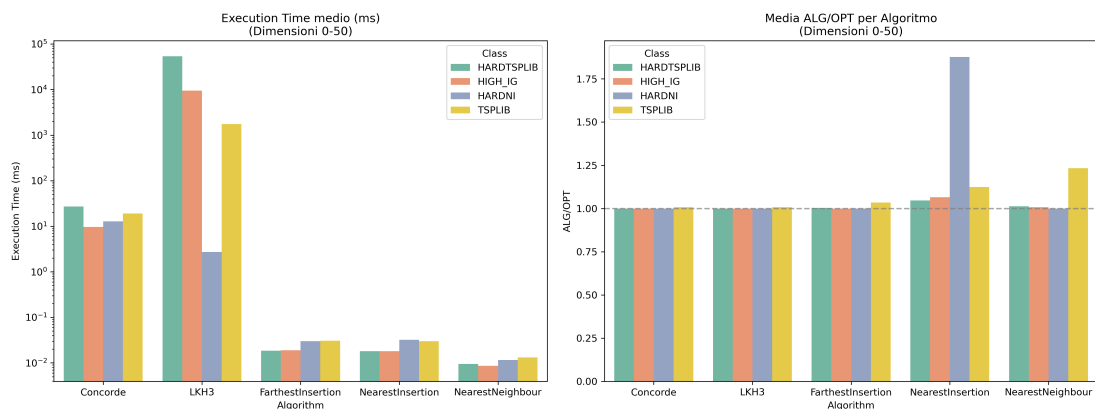


Figura 5.1: Performance aggregata dei solver su istanze di dimensione compresa tra 0 e 50 nodi.

5.6.1.2 Dimensioni medie (51-200 nodi)

In questa fascia si iniziano a delineare le prime differenze significative tra gli algoritmi. LKH3 mostra tempi di esecuzione proibitivi ($\approx 10^5$ ms) che ne limitano drasticamente l'applicabilità pratica, nonostante la buona qualità delle soluzioni prodotte.

È interessante notare come diverse classi di istanze mostrino comportamenti differenziati, suggerendo che la struttura intrinseca del problema influenzi significativamente le prestazioni algoritmiche.

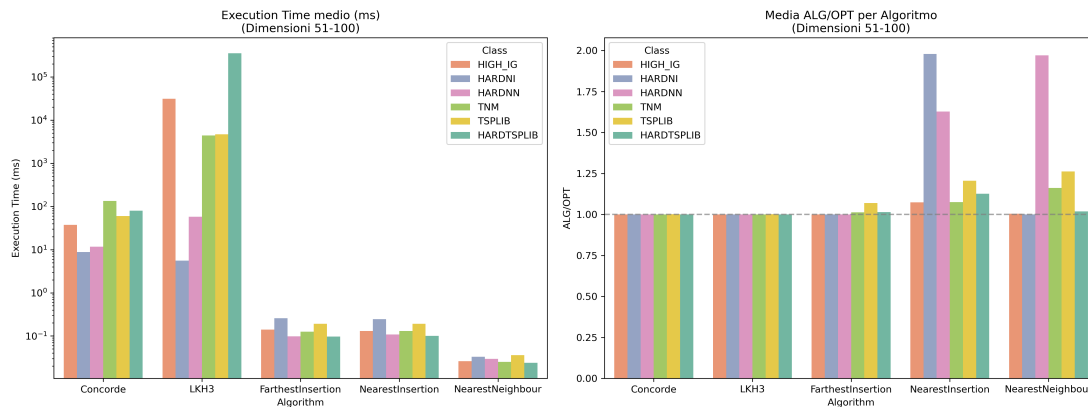


Figura 5.2: Performance aggregata dei solver su istanze di dimensione compresa tra 51 e 100 nodi.

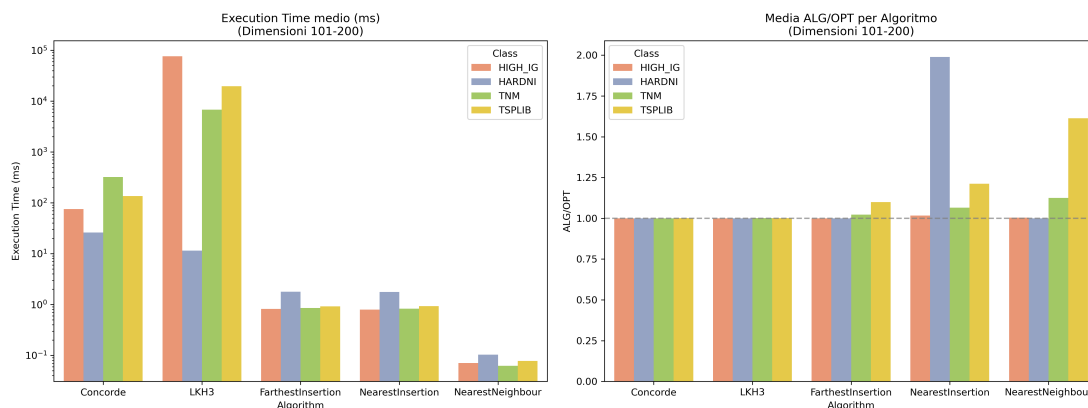


Figura 5.3: Performance aggregata dei solver su istanze di dimensione compresa tra 101 e 200 nodi.

5.6.1.3 Dimensioni elevate (201-2000 nodi)

E' sorprendente vedere come, nonostante essendo delle istanze "generiche", quelle appartenenti alla classe *TSPLIB* riescono a mettere in difficoltà risolutori come *Concorde* e *LKH3*, portando a tempi di esecuzioni maggiori e approssimazioni peggiori rispetto alle istanze con alto integrality-gap.

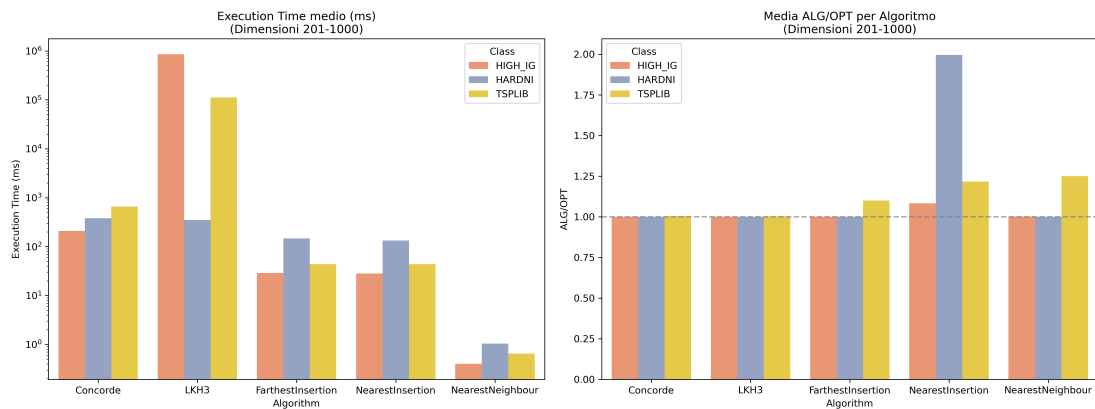


Figura 5.4: Performance aggregata dei solver su istanze di dimensione compresa tra 201 e 1000 nodi.

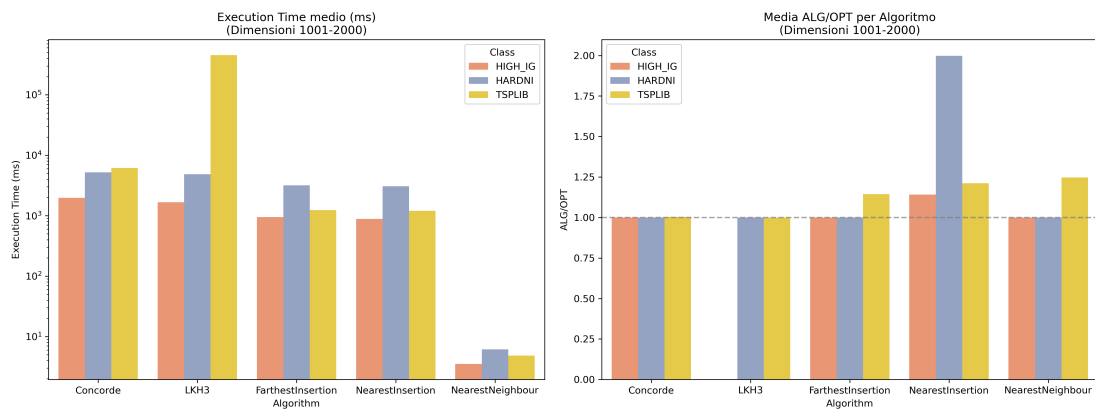


Figura 5.5: Performance aggregata dei solver su istanze di dimensione compresa tra 1001 e 2000 nodi.

5.6.1.4 Dimensioni molto elevate (2001+ nodi)

Nelle fasce dimensionali più ampie, la selezione algoritmica diventa cruciale. Osservando i tempi di esecuzione, le istanze TSPLIB e NN risultano simili per tutti i risolutori, suggerendo quindi che è la grandezza delle istanze a rappresentare la vera difficoltà per gli algoritmi.

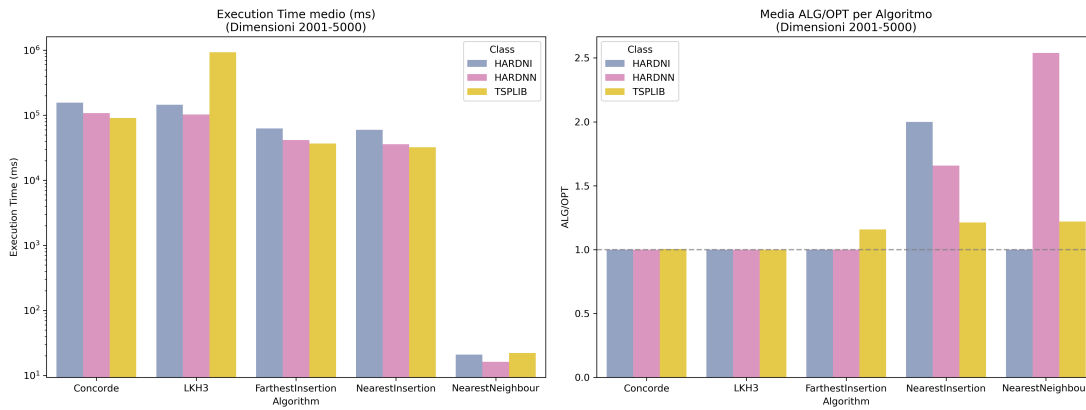


Figura 5.6: Performance aggregata dei solver su istanze di dimensione compresa tra 2001 e 5000 nodi.

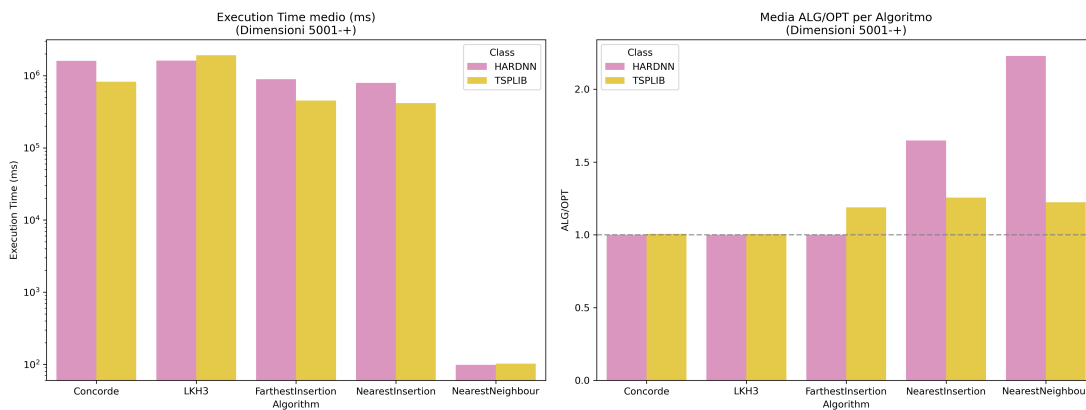


Figura 5.7: Performance aggregata dei solver su istanze di dimensione a partire da 5001 nodi.

5.6.2 Analisi Comparativa su Classi Specifiche

5.6.2.1 Classe HARDNI

Le istanze della classe HARDNI rappresentano un caso di studio particolarmente interessante per valutare la robustezza degli algoritmi. Come evidenziato nella Figura 5.8, si osserva come il risolutore `NearestInsertion` mostri prestazioni significativamente inferiori rispetto agli altri algoritmi, con un rapporto ALG/OPT che tende asintoticamente a 2.0 indipendentemente dalla dimensione del problema. Questo comportamento è proprio quello che ci aspettiamo: le istanze sono costruite per avere esattamente questo comportamento, garantito teoricamente, e rappresentano un punto critico per le euristiche di inserimento. In contrasto, `Concorde` e `LKH3` mantengono prestazioni ottimali ($ALG/OPT = 1.0$), confermando la loro natura di algoritmi esatti e di miglioramento rispettivamente. Tuttavia, come visibile nel grafico, presentano una curva dei tempi di esecuzione esponenziale che li rende proibitivi per istanze superiori a 1000 nodi (tempi $> 10^5$ ms). Da notare come `Concorde`

mostrì un comportamento particolare, con tempi di esecuzione maggiori rispetto a LKH3 per istanze di piccole dimensioni, ma una scalabilità leggermente migliore per istanze più grandi.

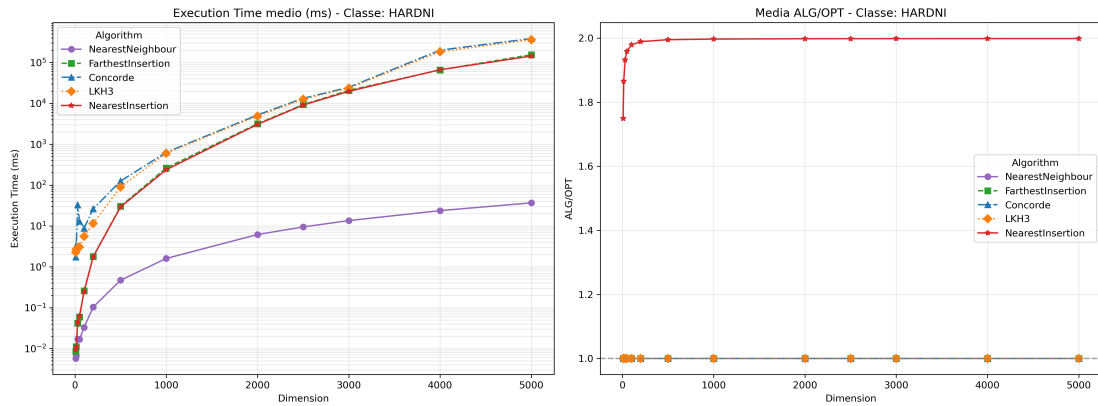


Figura 5.8: Confronto delle prestazioni sulla classe HARDNI. Si noti la divergenza tra algoritmi esatti ed euristici sia in termini di qualità (alto) che di tempo di esecuzione (basso).

5.6.2.2 Classe HARDNN

La classe HARDNN rivela un comportamento peculiare dell'algoritmo NearestNeighbour, come mostrato nella Figura 5.34. L'ALG/OPT mostra un andamento non monotono, con un picco di 2.7 a dimensione 4000 nodi per poi ridiscendere a 2.2, suggerendo una particolare sensibilità alla struttura del problema a medie dimensioni.

Interessante notare come NearestInsertion, pur mantenendo una prestazione più stabile ($ALG/OPT \approx 1.65$), dimostri comunque difficoltà con questa classe di problemi. Tutti gli algoritmi mostrano una crescita temporale simile, raggiungendo 10^6 ms per le istanze più grandi, sebbene con differenze significative nella qualità delle soluzioni ottenute.

Nella sezione sulle istanze peggiori, avevamo osservato che il rapporto ALG/OPT può crescere approssimativamente come $\log(n)$ (come dimostrato da Rosenkratz et al.). Tuttavia, qui non si osserva lo stesso comportamento: i dati sono ottenuti su più esecuzioni con seed diversi, e partendo da nodi casuali il rapporto medio non peggiora tanto quanto nel caso peggiore teorico.

È interessante sottolineare che, nelle istanze HARDNN, anche osservando più esecuzioni casuali, il comportamento peggiore si verifica comunque. Questo perché le istanze difficili per NearestNeighbour sono altamente simmetriche, rendendo praticamente irrilevante il nodo iniziale.

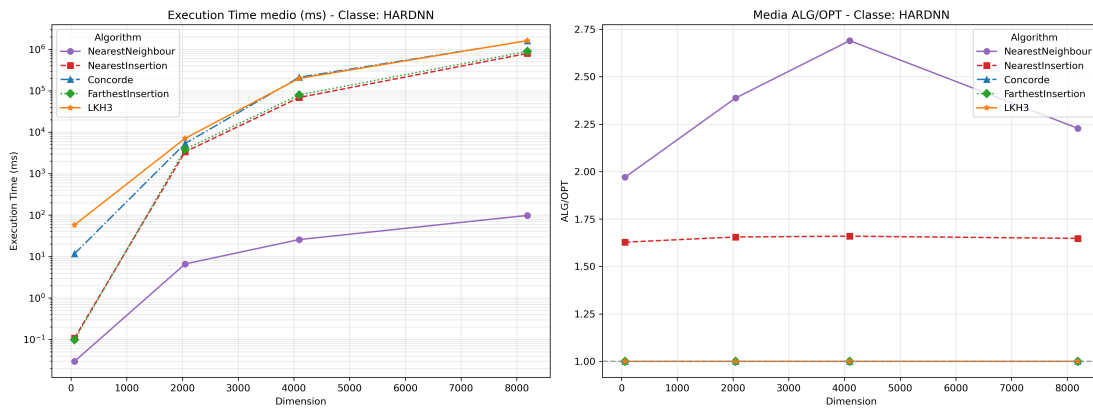


Figura 5.9: Prestazioni sulla classe HARDNN. Evidente il comportamento non monotono del Nearest Neighbour (linea rossa) nella metrica ALG/OPT.

5.6.2.3 Classe HIGH_IG

Le istanze con alto integrality gap (HIGH_IG) rappresentano una sfida particolare per gli algoritmi TSP. Come visibile nella Figura 5.10, la maggior parte dei risolutori mantiene prestazioni accettabili ($ALG/OPT \approx 1.0$), con l'eccezione notevole di NearestInsertion che mostra alta variabilità e prestazioni inferiori, come meglio dettagliato nella Figura 5.11.

Un risultato sorprendente è il comportamento di LKH3, che raggiunge picchi di 10^6 ms a 600 nodi per poi mostrare un miglioramento inatteso a dimensioni maggiori. Questo pattern suggerisce la possibile presenza di meccanismi di ottimizzazione interna che si attivano superata una certa soglia di complessità.

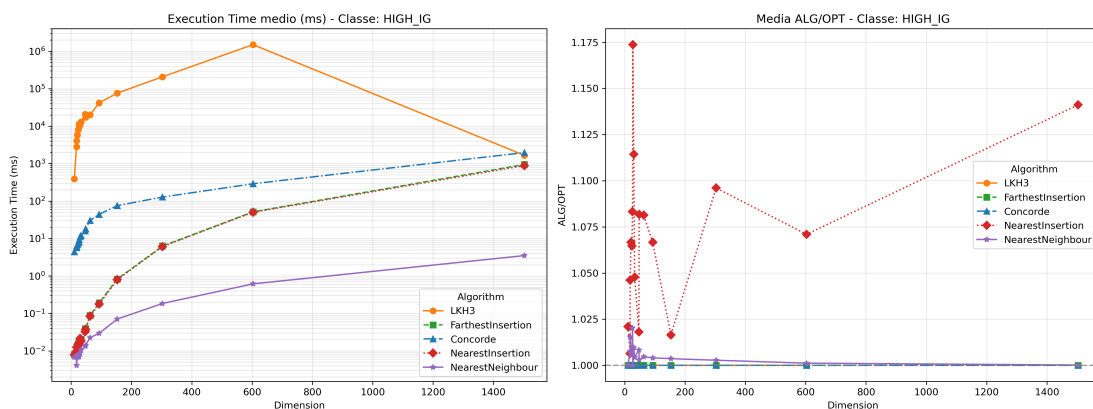


Figura 5.10: Prestazioni sulla classe HIGH_IG. Si osservi l'anomalia temporale di LKH3 (linea verde) e le prestazioni variabili di NearestInsertion (linea viola).

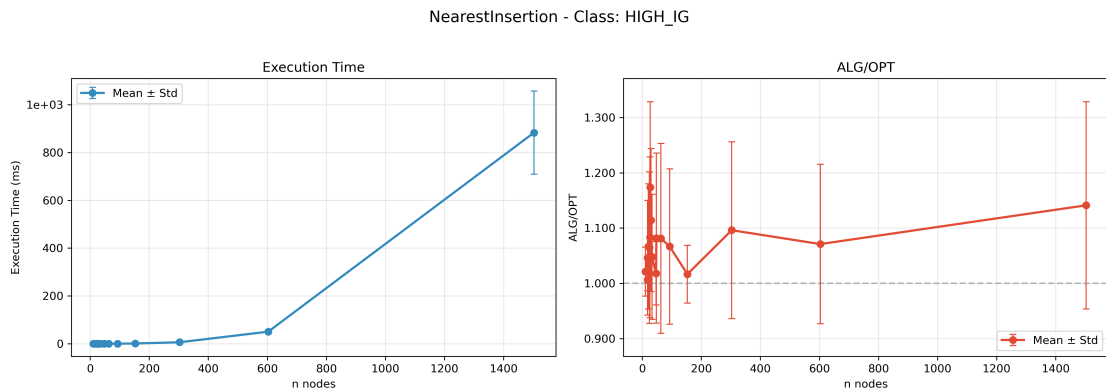


Figura 5.11: Dettaglio delle prestazioni di Nearest Insertion su HIGH_IG. Le ampie barre di errore evidenziano l'alta variabilità dei risultati.

5.6.2.4 Classe TSPLIB

La classe TSPLIB è l'unica nel dataset a contenere istanze che raggiungono e superano i 7000 nodi, come nel caso dell'istanza *pla7397*. Questo consente un'analisi del comportamento degli algoritmi su scala molto ampia, rivelando punti di forza e limiti.

Il comportamento del *NearestNeighbour* è particolarmente anomalo: in presenza di istanze molto piccole (es. *burma14*, con solo 14 nodi), si osserva un valore di approssimazione ALG/OPT estremamente elevato, pari a 8.99. Questo indica una grave inefficienza dell'euristica in scenari di piccola dimensione. Tuttavia, all'aumentare della dimensione, il valore si stabilizza su intervalli più accettabili (circa 1.4–1.6), seppur rimanendo il peggiore tra gli algoritmi considerati.

Gli altri algoritmi (*FarthestInsertion*, *NearestInsertion*, *LKH3*, *Concorde*) mantengono valori di approssimazione mediamente compresi tra 1.2 e 1.4, suggerendo una maggiore robustezza rispetto a *NearestNeighbour*, anche su istanze di grandi dimensioni.

Dal punto di vista dei tempi di esecuzione, *Concorde*, *LKH3* e *FarthestInsertion* impiegano tempi molto simili sulle istanze più grandi, ma è *LKH3* a registrare il tempo medio più elevato, con oltre **3.9 secondi** su *f13795*, seguita da *Concorde* con oltre **1.2 secondi** su *pla7397*. Questo comportamento suggerisce una maggiore complessità computazionale in *LKH3* su istanze di scala molto ampia.

La figura 5.17 (approssimazione) e la figura 5.15 (tempo di esecuzione) mostrano un comportamento coerente tra i vari algoritmi. In particolare, evidenziano come le istanze più grandi siano trattate in modo simile da parte di tutti i risolutori, a eccezione di *NearestNeighbour*, che rimane nettamente inferiore in termini di qualità della soluzione.

Un'ultima osservazione importante riguarda il fatto che, per ogni risolutore, esiste almeno un'istanza TSPLIB che mette in difficoltà l'algoritmo, sia in termini di tempo di esecuzione che di approssimazione. Riassumendo:

- **Concorde**: peggior tempo su *pla7397* (1.22s), peggior ratio su *burma14* (1.0548)

- **LKH3**: peggior tempo su f13795 (3.91s), peggior ratio su burma14 (1.0548)
- **FarthestInsertion**: peggior tempo su pla7397 (0.65s), peggior ratio su brg180 (1.6051)
- **NearestInsertion**: peggior tempo su pla7397 (0.62s), peggior ratio su brg180 (1.4554)
- **NearestNeighbour**: peggior tempo su pla7397 (0.11s), ma peggior ratio su brg180 (8.9944)

Queste osservazioni mostrano chiaramente come la dimensione dell'istanza e la sua struttura interna (anche se della stessa classe) possano influenzare drasticamente le performance di ogni algoritmo.

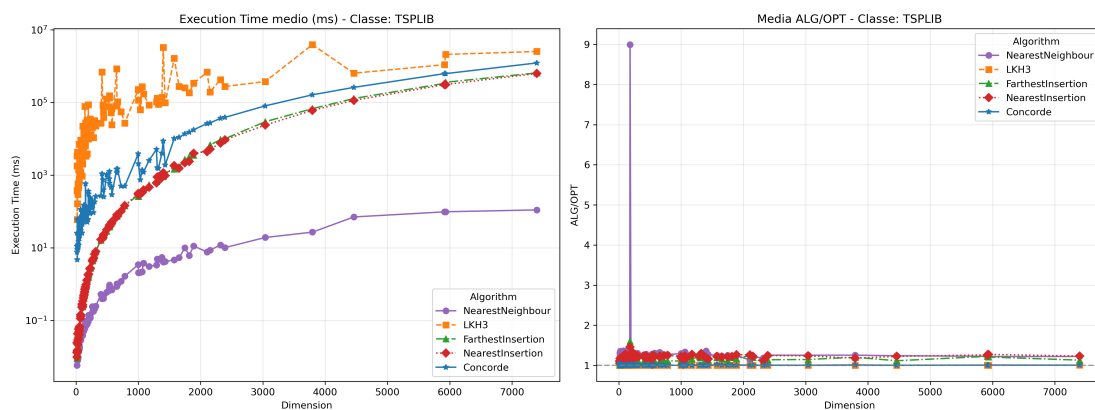


Figura 5.12: Performance aggregata dei solver su istanze TSPLIB.

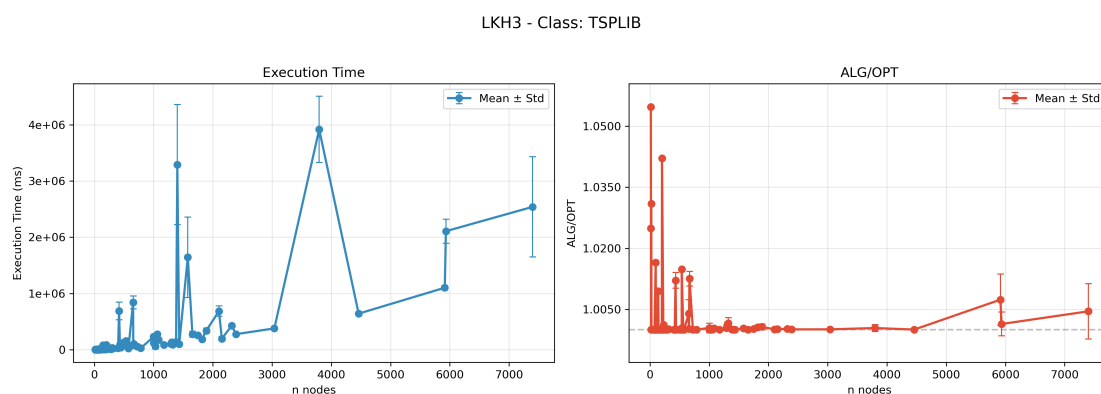


Figura 5.13: Performance LKH3 su istanze TSPLIB.

Concorde - Class: TSPLIB

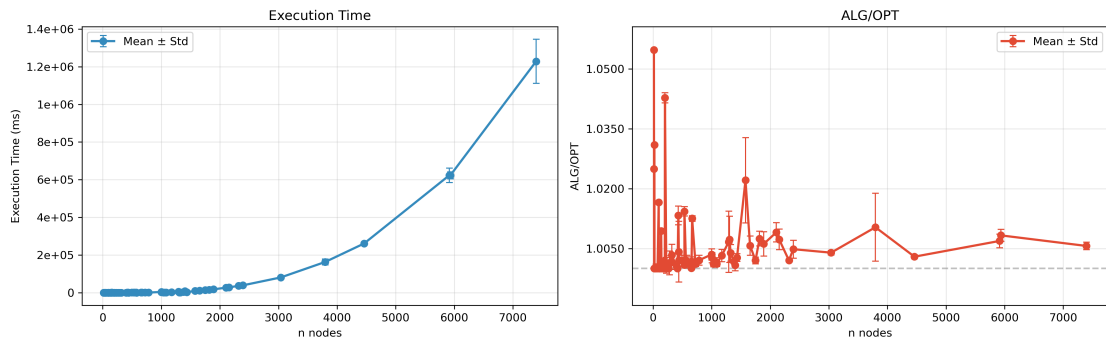


Figura 5.14: Performance Concorde su istanze TSPLIB.

FarthestInsertion - Class: TSPLIB

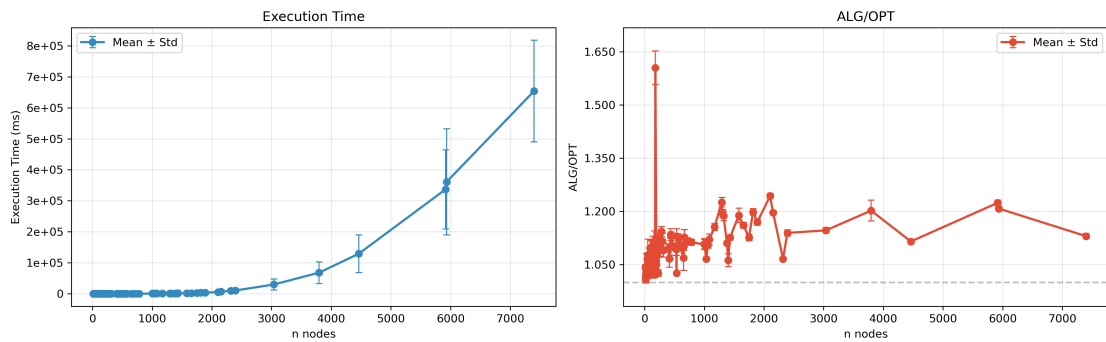


Figura 5.15: Performance Farthest Insertion su istanze TSPLIB.

NearestNeighbour - Class: TSPLIB

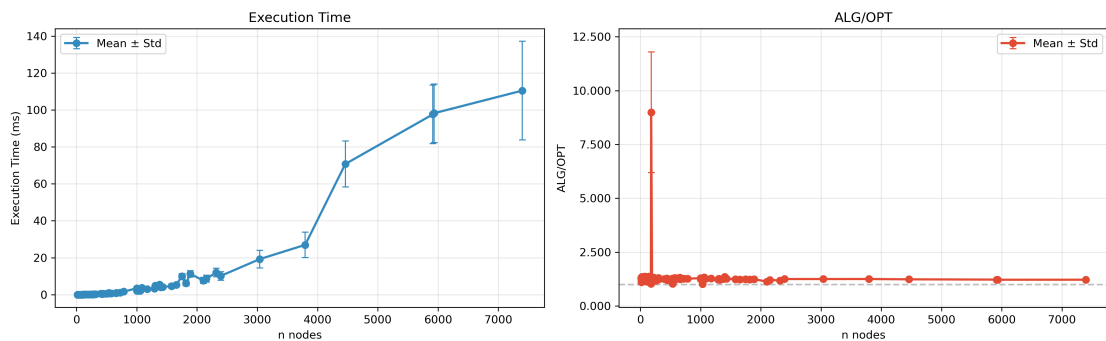


Figura 5.16: Performance Nearest Neighbour su istanze TSPLIB.

5.6.2.5 Classe HARDTSPLIB

Concorde e *LKH3*, figure 5.19 e 5.20, si distinguono per la loro precisione, mantenendo un rapporto di approssimazione estremamente vicino all'ottimo (ALG/OPT 1.0). Tuttavia, questa accuratezza ha un costo in termini di tempo: all'aumentare della dimensione delle

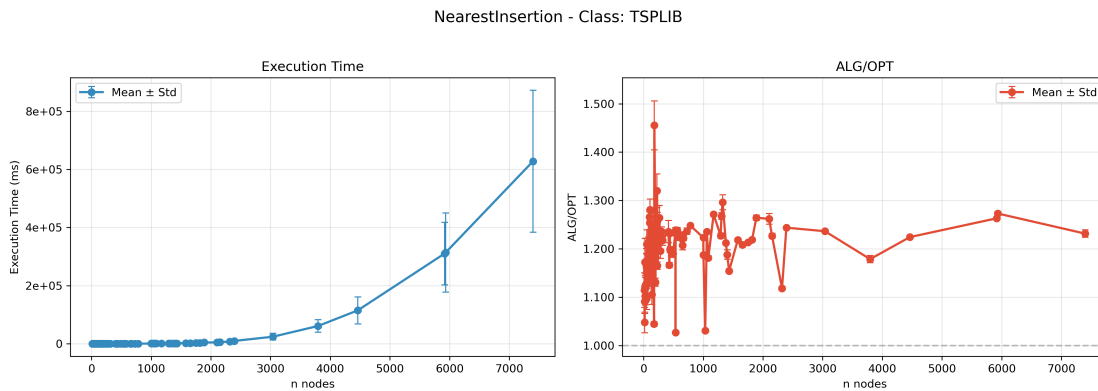


Figura 5.17: Performance Nearest Insertion su istanze TSPLIB.

istanze, i loro tempi di esecuzione crescono in modo significativo, rendendoli meno adatti a scenari in cui la velocità è un requisito critico.

Gli algoritmi euristici, come *FarthestInsertion* e *NearestInsertion* (figure 5.23 e 5.21), presentano un compromesso più equilibrato. Pur non raggiungendo la precisione di *Concorde* o *LKH3*, mantengono un rapporto di approssimazione accettabile (intorno a 1.1–1.4) e tempi di esecuzione contenuti, dimostrandosi adatti a problemi di media dimensione dove è necessaria una buona soluzione in tempi ragionevoli.

NearestNeighbour, Figura 5.22, rappresenta l'estremo opposto: è l'algoritmo più veloce, con tempi di esecuzione quasi trascurabili anche per istanze di 70 nodi, ma la qualità delle sue soluzioni è inferiore, come evidenziato dal rapporto di approssimazione più alto (fino a 1.2). Questo lo rende utile in contesti in cui è prioritario ottenere una soluzione rapida, anche se non ottimale.

Nota: quanto osservato qui (*tempi rapidi, ALG/OPT più alto*) corrisponde alle nostre analisi sperimentali in questa tesi. Il fatto che *NearestNeighbour* abbia prestazioni di questo tipo è anche previsto teoricamente nella letteratura esistente, poiché l'algoritmo è noto per essere veloce ma meno accurato.

Un aspetto particolarmente interessante è la scalabilità. Mentre gli algoritmi esatti tendono a diventare computazionalmente onerosi per istanze più grandi, gli euristici mostrano una crescita più graduale nei tempi di esecuzione. Questo suggerisce che, per problemi di dimensioni molto elevate, potrebbe essere preferibile optare per un approccio euristico, a meno che non sia strettamente necessaria una soluzione ottimale.

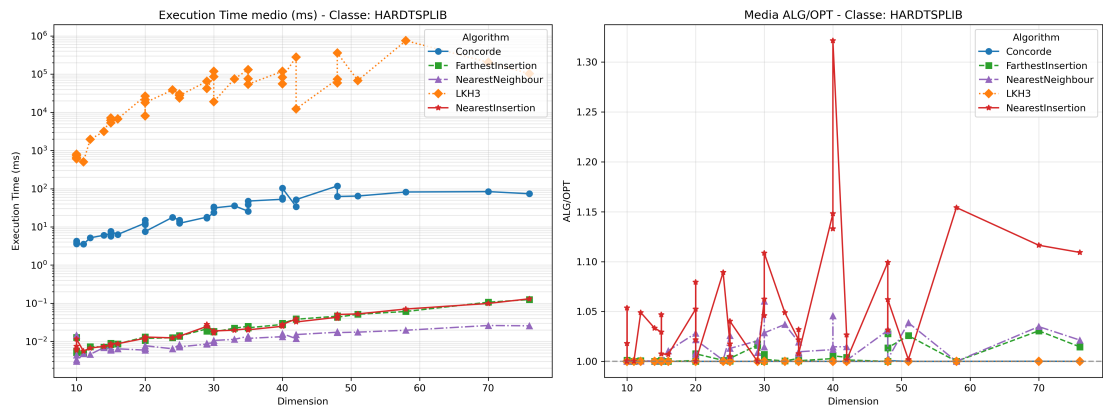


Figura 5.18: Performance aggregata dei solver su istanze HARDTSPLIB.

Concorde - Class: HARDTSPLIB

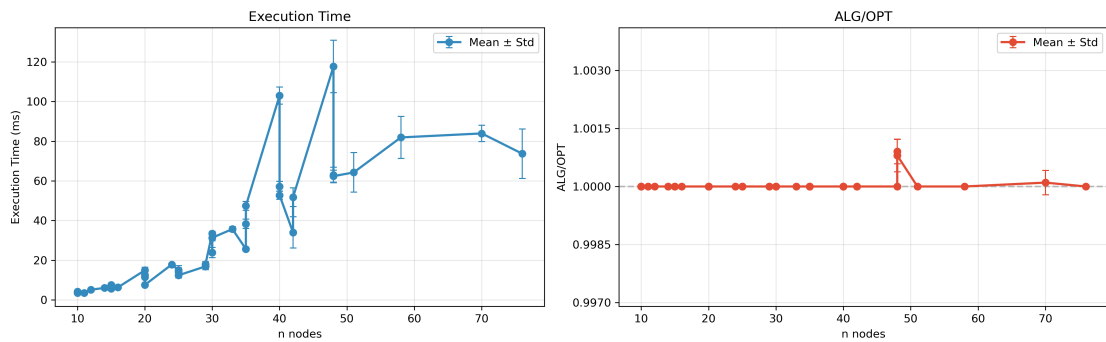


Figura 5.19: Performance Concorde su istanze HARDTSPLIB.

LKH3 - Class: HARDTSPLIB

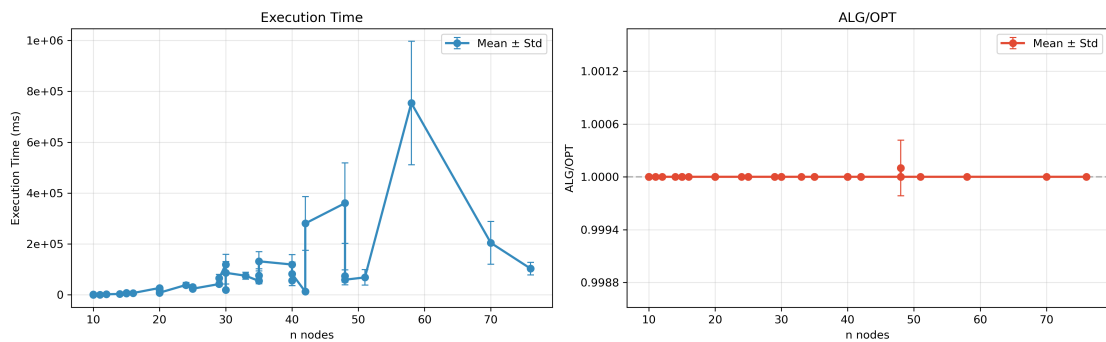


Figura 5.20: Performance LKH3 su istanze HARDTSPLIB.

5.6.2.6 Classe TNM

L'analisi delle istanze TNM rivela interessanti dinamiche nelle prestazioni degli algoritmi TSP. Come evidenziato nella Figura 5.25, *Concorde* mantiene un'eccezionale stabilità nella

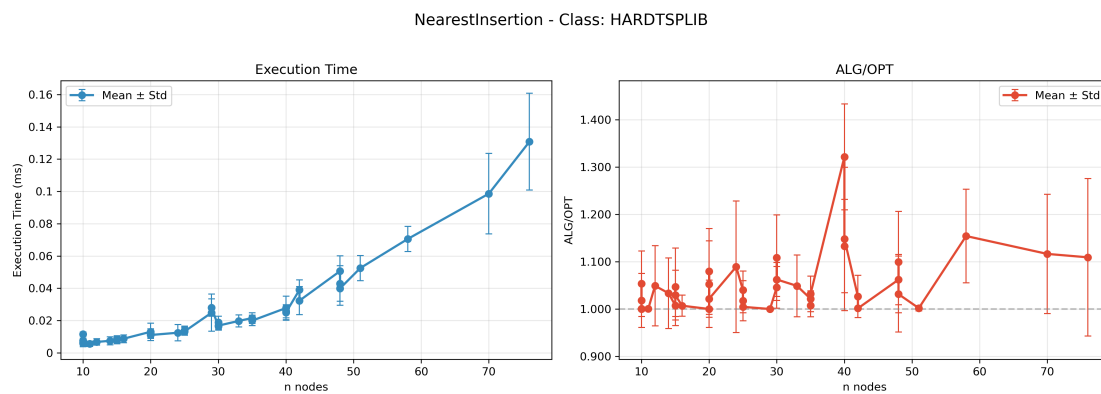


Figura 5.21: Performance Nearest Insertion su istanze HARDTSPLIB.

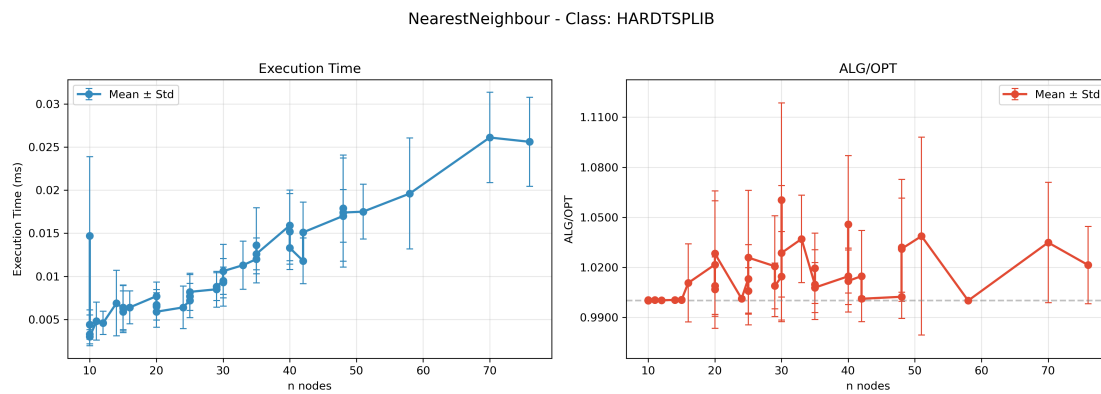


Figura 5.22: Performance Nearest Neighbour su istanze HARDTSPLIB.

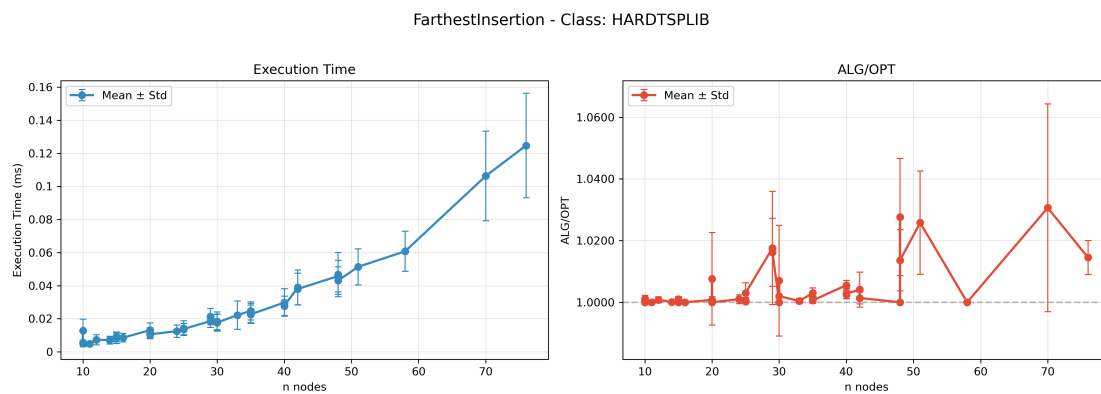


Figura 5.23: Performance Farthest Insertion su istanze HARDTSPLIB.

qualità delle soluzioni. Tuttavia, i tempi di esecuzione mostrano una crescita lineare da 400ms a 600ms all'aumentare della dimensione del problema.

LKH3 (Figura 5.26) presenta caratteristiche simili in termini di precisione, ma con un comportamento computazionale radicalmente diverso: i tempi di esecuzione subiscono un in-

cremento esponenziale, passando da 2 secondi per 60 nodi a ben 12 secondi per 200 nodi. Questo rende l'algoritmo poco pratico per istanze di grandi dimensioni nonostante la qualità delle soluzioni.

Le euristiche costruttive mostrano un diverso equilibrio tra qualità e velocità. *FarthestInsertion* (Figura 5.29) si distingue per il miglior compromesso, con ALG/OPT compreso tra 1.04 e 1.12 e tempi di esecuzione sempre inferiori a 2ms. *NearestInsertion* (Figura 5.27), sebbene simile in termini di velocità, presenta una maggiore variabilità nei risultati (ALG/OPT 1.04-1.20), come evidenziato dalle barre di errore più ampie.

NearestNeighbour (Figura 5.28) rappresenta l'estremo opposto: con tempi di esecuzione quasi istantanei (<0.1 ms) ma qualità delle soluzioni significativamente inferiore (ALG/OPT fino a 1.24). Questi risultati sono sintetizzati nella Figura 5.24, che mostra chiaramente il trade-off tra precisione e velocità per i diversi algoritmi.

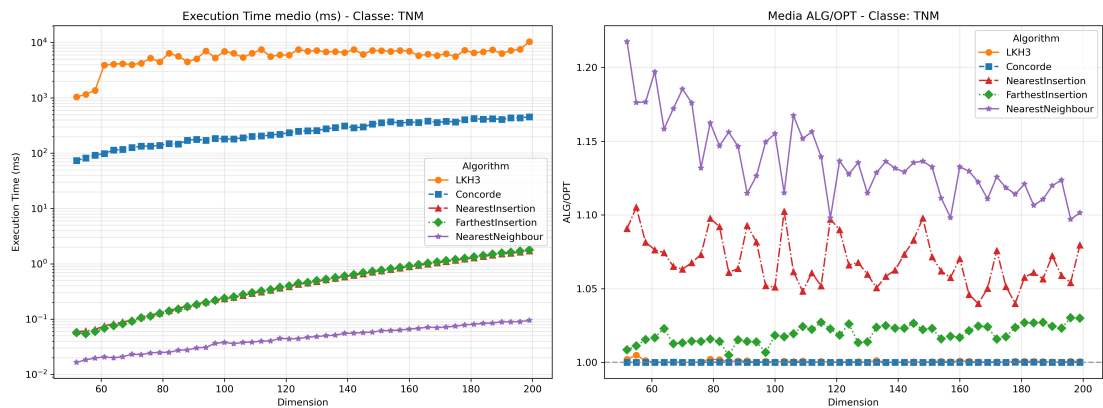


Figura 5.24: Confronto aggregato delle prestazioni degli algoritmi sulle istanze TNM. I tempi di esecuzione (sinistra) e i valori di ALG/OPT (destra) mostrano il trade-off tra precisione e velocità.

Concorde - Class: TNM

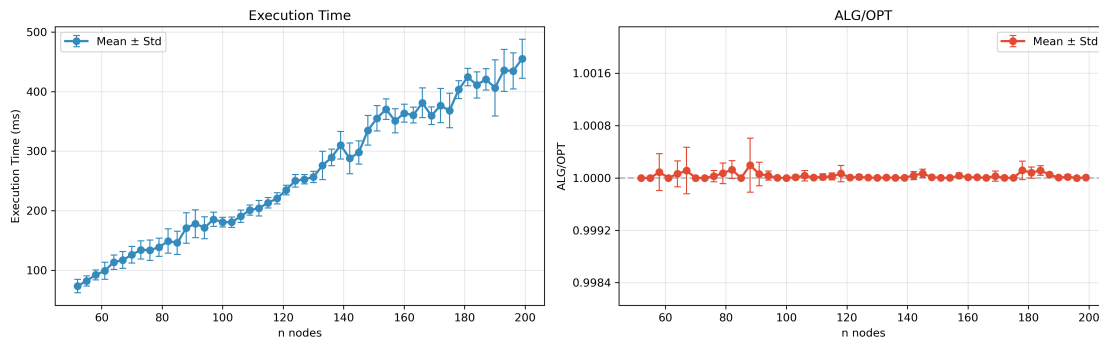


Figura 5.25: Prestazioni di Concorde su istanze TNM. Si noti la stabilità dei valori ALG/OPT (alto) e la crescita lineare dei tempi (basso).

LKH3 - Class: TNM

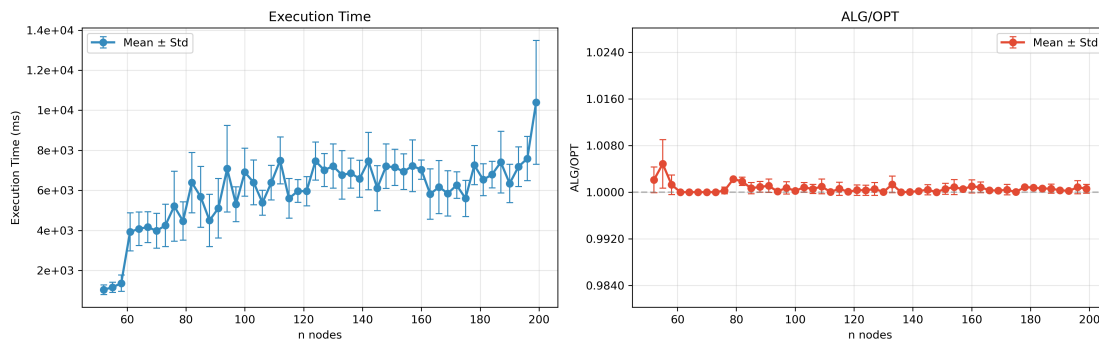


Figura 5.26: Prestazioni di LKH3 su istanze TNM. Osservare l'andamento esponenziale dei tempi di esecuzione (basso) nonostante la precisione ottimale (alto).

NearestInsertion - Class: TNM

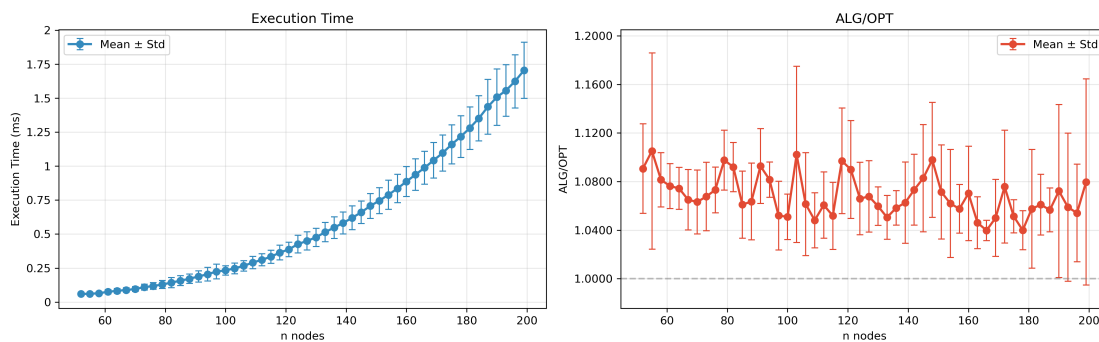


Figura 5.27: Prestazioni di Nearest Insertion su istanze TNM. Notevole la maggiore variabilità nei valori ALG/OPT rispetto a FarthestInsertion.

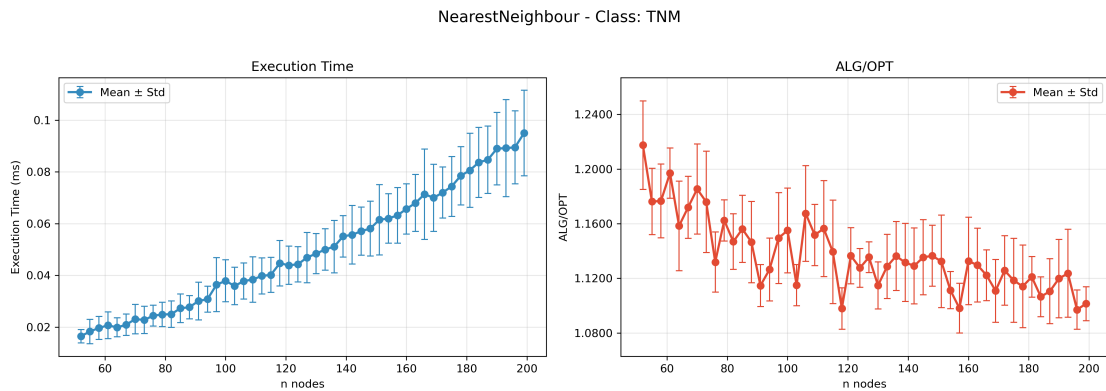


Figura 5.28: Prestazioni di Nearest Neighbour su istanze TNM. Tempi di esecuzione trascurabili (sinistra) ma qualità delle soluzioni inferiori (destra).

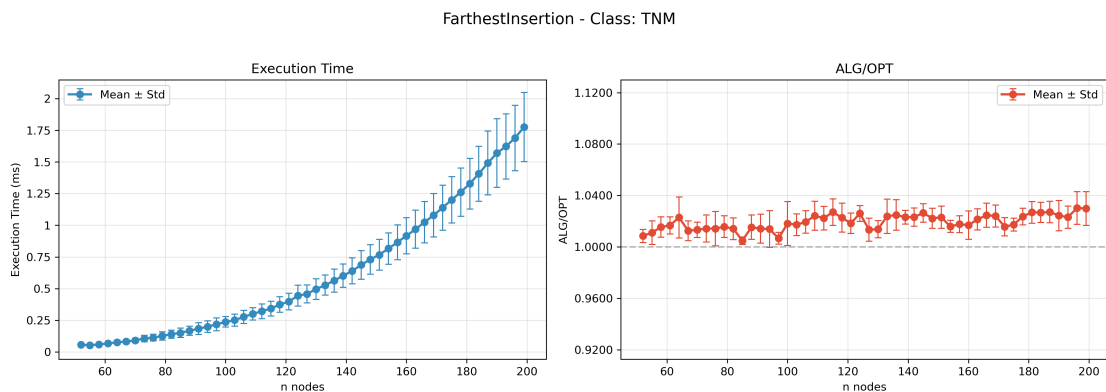


Figura 5.29: Prestazioni di Farthest Insertion su istanze TNM. Si apprezzi il miglior compromesso tra qualità e velocità tra le euristiche.

5.6.3 Analisi Comparativa per Algoritmo

5.6.3.1 Concorde

Come evidenziato nella Figura 5.30, Concorde conferma la sua reputazione di algoritmo esatto, mantenendo un rapporto ALG/OPT estremamente vicino a 1.0 (1.00-1.05) su tutte le classi di istanze e dimensioni. Tuttavia, questa precisione ha un prezzo computazionale significativo:

- Per istanze piccole (fino a 1000 nodi), i tempi rimangono accettabili ($10^2 - 10^3$ ms)
- Oltre i 4000 nodi, la crescita diventa esponenziale, raggiungendo 10^6 ms

Particolarmente interessante è il comportamento su HIGH_IG, dove nonostante l'alto integrality gap, Concorde mantiene la sua precisione ottimale, dimostrando robustezza teorica.

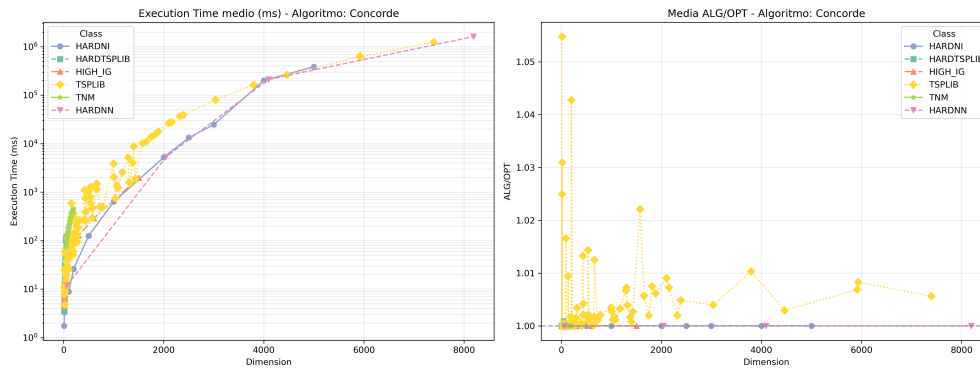


Figura 5.30: Prestazioni di Concorde su tutte le classi di istanze. Notare la scala logaritmica per i tempi di esecuzione (sinistra) e la stabilità dell'ALG/OPT (destra).

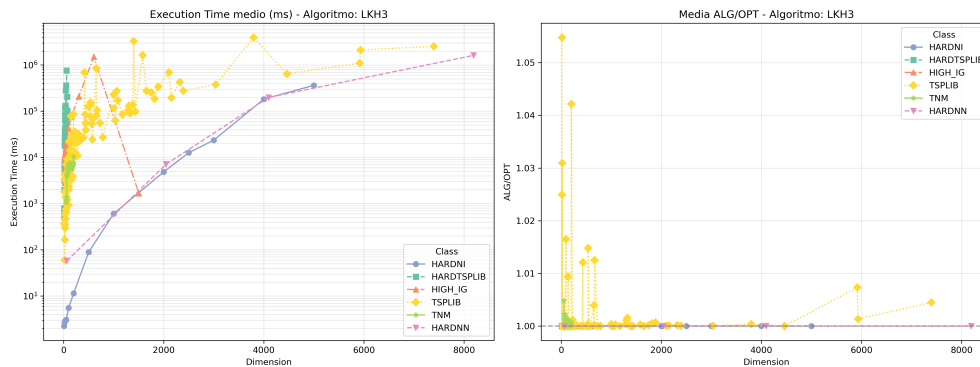


Figura 5.31: Prestazioni di LKH3. Si notino le oscillazioni nei tempi di esecuzione non correlate alla dimensione.

5.6.3.2 LKH3

LKH3 (Figura 5.31) mostra caratteristiche uniche:

- Precisione paragonabile a Concorde (ALG/OPT 1.00-1.05)
- Tempi di esecuzione con andamento non monotono, particolarmente evidente su HIGH_IG e TNM
- Picchi improvvisi ($> 10^4$ ms) anche per istanze medie (600-1000 nodi)

Questo comportamento suggerisce che LKH3, pur essendo un algoritmo eccellente, possa essere sensibile a particolari strutture del grafo più che alla semplice dimensione del problema.

5.6.3.3 FarthestInsertion

L'analisi in Figura 5.32 rivela perché FarthestInsertion è spesso considerato il miglior algoritmo euristico:

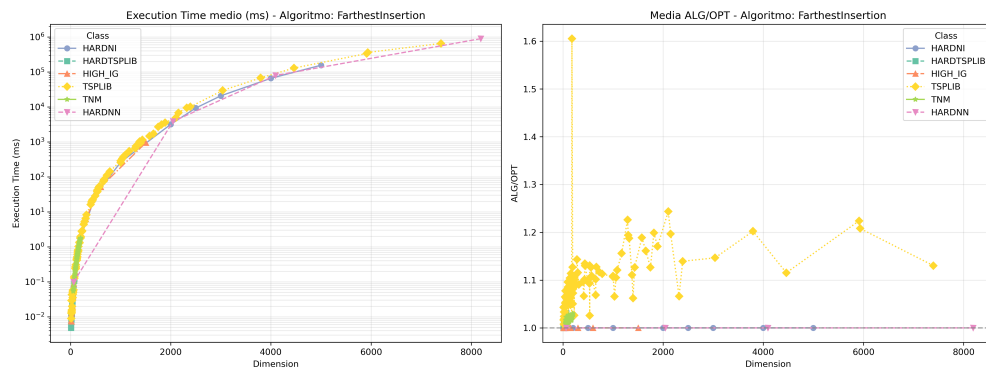


Figura 5.32: Prestazioni di FarthestInsertion. Costante rapporto qualità/velocità.

- Tempi di esecuzione contenuti ($10^0 - 10^2$ ms) anche per 8000 nodi
- ALG/OPT stabile, con un'unica eccezione per TSPLIB (fino a 1.6)
- Crescita temporale quasi lineare, non esponenziale

Particolarmente degno di nota è il comportamento su TSPLIB, dove raggiunge $ALG/OPT=1.2$ con tempi inferiori a 10ms, rendendolo ideale per applicazioni real-time.

5.6.3.4 NearestInsertion

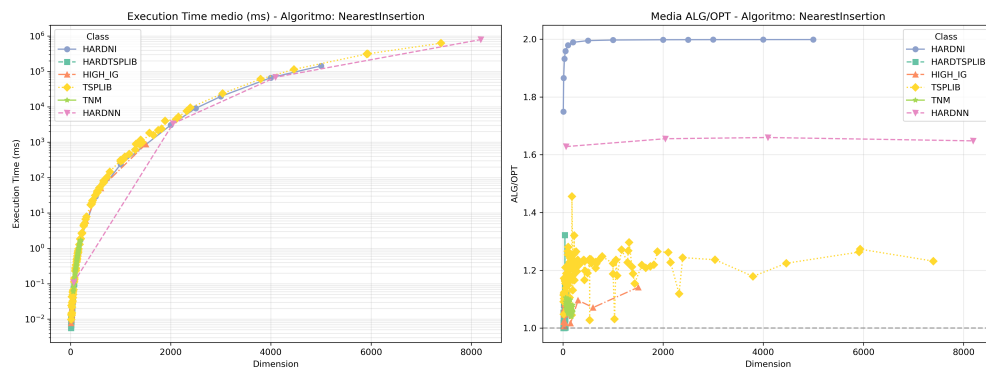


Figura 5.33: Prestazioni di NearestInsertion. Notevole variabilità nell'ALG/OPT.

Come mostrato in Figura 5.33, NearestInsertion presenta:

- La maggiore variabilità tra gli algoritmi (ALG/OPT 1.0-1.8)
- Prestazioni particolarmente scadenti su HARDNI e HIGH_IG
- Tempi di esecuzione simili a FarthestInsertion ma qualità inferiore

Questi risultati suggeriscono che la strategia di inserimento "più vicino" sia intrinsecamente più sensibile alla struttura del problema rispetto all'approccio "più lontano".

5.6.3.5 NearestNeighbour

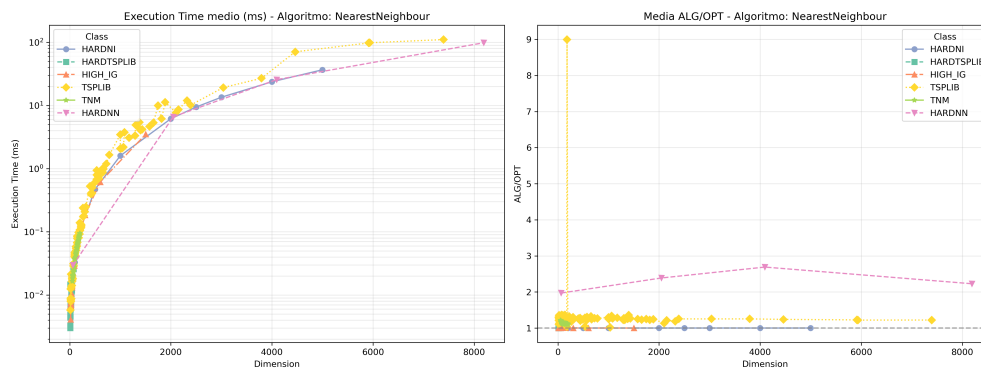


Figura 5.34: Prestazioni di NearestNeighbour. Tempi trascurabili ma qualità compromessa.

L'analisi in Figura 5.34 evidenzia le caratteristiche uniche di NearestNeighbour:

- Tempi di esecuzione inferiori a 1 ms ($10^{-1} - 10^{-2}$ ms) per tutte le dimensioni
- ALG/OPT più variabile (1.0-2.0) tra le classi di problemi
- Prestazioni peggiori su HARDNI e HARDNN, migliori su TNM e TSPLIB

Questi risultati confermano il ruolo di NearestNeighbour come algoritmo "quick-and-dirty", utile per ottenere soluzioni approssimate in tempo trascurabile quando la qualità non è prioritaria.

Capitolo 6

Conclusione

6.1 Considerazioni Finali

L'analisi approfondita dei risultati ha rivelato diversi pattern significativi che meritano una riflessione critica. Contrariamente a quanto spesso assunto, la dimensione del problema da sola non spiega completamente il comportamento degli algoritmi. Sebbene rappresenti certamente un fattore importante, abbiamo osservato che:

- La **struttura interna** delle istanze gioca un ruolo altrettanto cruciale. Istanze della stessa dimensione ma di classi diverse (ad esempio HIGH_IG vs TSPLIB) mostrano variazioni di prestazione per lo stesso algoritmo.
- Esistono **soglie critiche** dove il rapporto qualità-prestazioni cambia drasticamente:
 - Sotto i 100 nodi: gli algoritmi esatti dominano incontrastati
 - Tra 100-500 nodi: le euristiche iniziano a mostrare il loro vantaggio
 - Oltre 1000 nodi: il divario prestazionale si amplifica
- Alcune **anomalie interessanti** sfidano le aspettative:
 - LKH3 a volte risolve istanze grandi più velocemente di quelle medie
 - FarthestInsertion mostra stabilità inattesa su HIGH_IG
 - NearestNeighbour ha prestazioni migliori su TNM che su TSPLIB

6.1.1 Implicazioni Pratiche

Alla luce di queste osservazioni, le linee guida per la scelta dell'algoritmo devono considerare:

- **Non solo la dimensione:** Un'istanza di 500 nodi HIGH_IG può essere più impegnativa di una TSPLIB da 800 nodi. La classificazione delle istanze è essenziale quanto la loro grandezza.

- **Contesto applicativo:** In scenari reali dove il tempo è critico ma serve una buona approssimazione, FarthestInsertion si rivela spesso la scelta migliore, superando anche alcuni algoritmi esatti per il bilanciamento offerto.
- **Risorse disponibili:** Per istanze oltre i 1000 nodi, la scelta tra LKH3 e euristiche dipende strettamente dall'hardware a disposizione e dalla tolleranza all'approssimazione.

6.1.2 Prospettive di Ricerca

I risultati suggeriscono diverse direzioni promettenti:

- **Ibridazione algoritmica:** Combinare la precisione di Concorde con la velocità di FarthestInsertion per istanze medie potrebbe superare i limiti degli approcci attuali.
- **Modelli predittivi:** Sviluppare modelli che, in base alle caratteristiche dell'istanza (non solo dimensione), possano prevedere l'algoritmo ottimale.
- **Ottimizzazioni specifiche:** I comportamenti anomali osservati (es. LKH3 su HI-GH_IG) indicano potenziali aree per miglioramenti mirati che potrebbero portare a breakthrough prestazionali.

In conclusione, questo studio non solo fornisce un quadro completo delle prestazioni algoritmiche, ma sfida alcuni assunti comuni nel campo. La complessa interazione tra dimensione, struttura delle istanze e caratteristiche algoritmiche emerge come un terreno fertile per ricerca futura, mentre le linee guida pratiche qui delineate offrono un solido riferimento per applicazioni reali. L'evidenza sperimentale raccolta suggerisce che il futuro della risoluzione del TSP potrebbe risiedere in approcci più sofisticati che considerino multidimensionalmente le caratteristiche del problema, piuttosto che affidarsi a metriche unidimensionali come la semplice grandezza dell'istanza.

Bibliografia e Sitografia

- [1] Daniel J. Rosenkrantz, Richard E. Stearns, and Philip M. Lewis. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.
- [2] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, Englewood Cliffs, NJ, 1982.
- [3] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. The traveling salesman problem: A computational study. https://www.researchgate.net/publication/268645053_The_Traveling_Salesman_Problem_A_Computational_Study, 2006.
- [4] Travelling salesman problem. https://en.wikipedia.org/wiki/Travelling_salesman_problem, 2025. Wikipedia, consultato il 17 giugno 2025.
- [5] J. A. Bondy and U. S. R. Murty. *Graph Theory*, volume 244 of *Graduate Texts in Mathematics*. Springer, London, 2008.
- [6] Douglas B. West. *Introduction to Graph Theory*. Prentice Hall, Upper Saddle River, NJ, 2nd edition, 2001.
- [7] Richard M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Springer, New York, 1972.
- [8] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, San Francisco, CA, 1979.
- [9] Ein alter Commis-Voyageur. *Der Handlungsreisende – wie er sein soll und was er zu tun hat, um Aufträge zu erhalten und eines glücklichen Erfolgs in seinen Geschäften gewiß zu sein – von einem alten Commis-Voyageur*. B.G. Teubner, Leipzig, 1832. The travelling salesman – how he must be and what he should do in order to get commissions and be sure of the happy success in his business – by an old commis-voyageur.

- [10] K. Menger. Bericht über ein mathematisches kolloquium. *Monatshefte für Mathematik und Physik*, 38:17–38, 1931.
- [11] George B. Dantzig, Ray Fulkerson, and Selmer Johnson. Solution of a large-scale traveling salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.
- [12] P. C. Gilmore and R. E. Gomory. A solvable case of the traveling salesman problem. *Proceedings of the National Academy of Sciences*, 51:178–181, 1964.
- [13] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC)*, pages 151–158. ACM, 1971.
- [14] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [15] Dexter C. Kozen. *Automata and Computability*. Springer, New York, 2012.
- [16] Wikipedia contributors. Decision problem — wikipedia, the free encyclopedia, 2024. Accessed: 2025-06-17.
- [17] Clay Mathematics Institute. Millennium prize problems. <https://www.claymath.org/millennium-problems>. Accessed: 2025-06-17.
- [18] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, Massachusetts, 1994.
- [19] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, Boston, MA, 3rd edition, 2012.
- [20] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.
- [21] Keld Helsgaun. An effective implementation of k-opt moves for the lin-kernighan tsp heuristic. Technical report, Computer Science, Roskilde University, DK-4000 Roskilde, Denmark, 2009. <http://webhotel4.ruc.dk/~keld/research/LKH/LKH-report.pdf>.
- [22] Keld Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. Technical report, Department of Computer Science, Roskilde University, DK-4000 Roskilde, Denmark, December 2017. http://webhotel4.ruc.dk/~keld/research/LKH-3/LKH-3_REPORT.pdf.

- [23] Gregory Gutin and Abraham P. Punnen. *The Traveling Salesman Problem and Its Variations*. Springer, 2006.
- [24] Eugene L. Lawler, Jan Karel Lenstra, A.H.G. Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Wiley, 1985.
- [25] F. Bock. An algorithm for solving “traveling-salesman” and related network optimization problems. Research report, Armour Research Foundation, 1958. Presented at the Operations Research Society of America Fourteenth National Meeting, St. Louis, October 24, 1958.
- [26] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6:791–812, 1958.
- [27] W. L. Eastman. *Linear Programming with Pattern Constraints*. PhD thesis, Department of Economics, Harvard University, Cambridge, Massachusetts, USA, 1958.
- [28] M. J. Rossman and R. J. Twery. A solution to the travelling salesman problem. In *Bulletin of the Operations Research Society of America Fourteenth National Meeting*, St. Louis, October 23–24 1958. Abstract E3.1.3.
- [29] J. D. C. Little, K. G. Murty, D. W. Sweeney, and C. Karel. An algorithm for the traveling salesman problem. *Operations Research*, 11(6):972–989, 1963.
- [30] Manfred Padberg and Giovanni Rinaldi. A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems. *SIAM Review*, 33(1):60–100, 1991.
- [31] Gerhard Reinelt. TSPLIB95: Library of Traveling Salesman Problems. Technical report, Universität Heidelberg, Institut für Informatik, 1995. <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/tsp95.pdf>.
- [32] Robert W. Floyd. Algorithm 97: Shortest path. *Communications of the ACM*, 5(6):345, 1962.
- [33] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1994.
- [34] Stefan Hougardy and Xianghui Zhong. Hard to solve instances of the euclidean traveling salesman problem. *Mathematical Programming Computation*, 13(3):587–603, 2021.

- [35] Paolo Vercesi, Stefano Gualandi, Monaldo Mastrolilli, and Luca M Gambardella. On the generation of metric tsp instances with a large integrality gap by branch-and-cut. *Mathematical Programming Computation*, 2023.
- [36] Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem, unknown. Accessed: 2025-07-22.
- [37] David S. Johnson and Lyle A. McGeoch. The traveling salesman problem: A case study in local optimization. In Emile H. L. Aarts and Jan Karel Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. Wiley, 1997.
- [38] George Dantzig, Ray Fulkerson, and Selmer Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2:393–410, 1954.
- [39] George Dantzig. Programming in a linear structure. Technical report, US Air Force Comptroller, USAF, Washington, D.C., USA, 1948.
- [40] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Pearson Education, 2017.
- [41] Daniel Benoit and Stephen Boyd. Finding the exact integrality gap for small traveling salesman problems. *Mathematics of Operations Research*, 33(4):921–932, 2008.

Ringrazio me stesso.

